VNUHCM - University of Science
Faculty of Information Technology

CS486 – Lecture 7

Locking

Content

- **Issues in concurrency control**
- Lock protocol
  - Introduction
  - Two-phase locking
  - Lock modes
  - Multiple granularity
  - Tree-base
- More on transaction processing

Issues in concurency control

- (1) Lost updated
- (2) Unrepeatable read
- (3) “Phantom”
- (4) Dirty read

Lost updated

- Examine

T_1	T_2
Read(A)	Read(A)
A:=A+10	A:=A+20
Write(A)	Write(A)

- T1 and T2 are performed concurrently

A=50	T_1	T_2
t_1	Read(A)	
t_2		Read(A)
t_3	A:=A+10	
t_4		A:=A+20
t_5	**Write(A)**	
t_6		**Write(A)**
	A=60	A=70

- The value of A modified at t_4 by T_1 is lost
- It is overwritten at t_6

What happens if T_2 writes an incorrect value ???

Unrepeatable read

- Examine

T_1	T_2
Read(A)	Read(A)
A:=A+10	Print(A)
Write(A)	Read(A)
	Print(A)

- T1 and T2 are performed concurrently

A=50	T_1	T_2	
t_1	Read(A)		
t_2		**Read(A)**	A=50
t_3	A:=A+10		
t_4		Print(A)	A=50
t_5	Write(A)		
t_6		**Read(A)**	A=60
t_7		Print(A)	A=60

- T_2 read A two times
- Results of these times are different

Phantom

- Examine T_1 and T_2 as follows
  - A and B are balances of banking accounts
  - T_1 transfers money from A to B
  - T_2 checks the sum of A and B

A=70, B=50	T_1	T_2	
t_1	Read(A)		A=70
t_2	A:=A-50		
t_3	**Write(A)**		A=20
t_4		Read(A)	A=20
t_5		Read(B)	B=50
t_6		**Print(A+B)**	A+B=70
t_7	Read(B)		
t_8	B:=B+50		
t_9	**Write(B)**		

Lost 50 ???

Dirty read

```
Transfer(account1, account2, money)
        1. Add money to account2
        2. Test if account1 has enough money
                2a. Not enough, remove money from account2
                2b. Enough, subtract money from account1
```

	T_1 transfers 150 from A_1 to A_2	T_2 transfer 250 from A_2 to A_3	A_1=100	A_2=200	A_3=300	
						600
t_1		$A_3:=A_3+250$			550	
t_2	$A_2:=A_2+150$			350		
t_3		Test $A_2 \geq 250$?				**Dirty read**
t_4	Test $A_1 \geq 150$?					
t_5		$A_2:=A_2-250$		100		
t_6	$A_2:=A_2-150$			-50		
						600

Dirty read

- Examine T_1 and T_2 as follows

	T_1	T_2
t_1	Read(A)	
t_2	A:=A+10	
t_3	Write(A)	
t_4		**Read(A)**
t_5		Print(A)
t_6	**Abort**	

- T_2 reads the value of A written by T_1
- But, T_1 aborts

Dirty read

Discussion

- A scheduler ***prevents***
  - Unserializable schedules
  - Issues in concurrency control

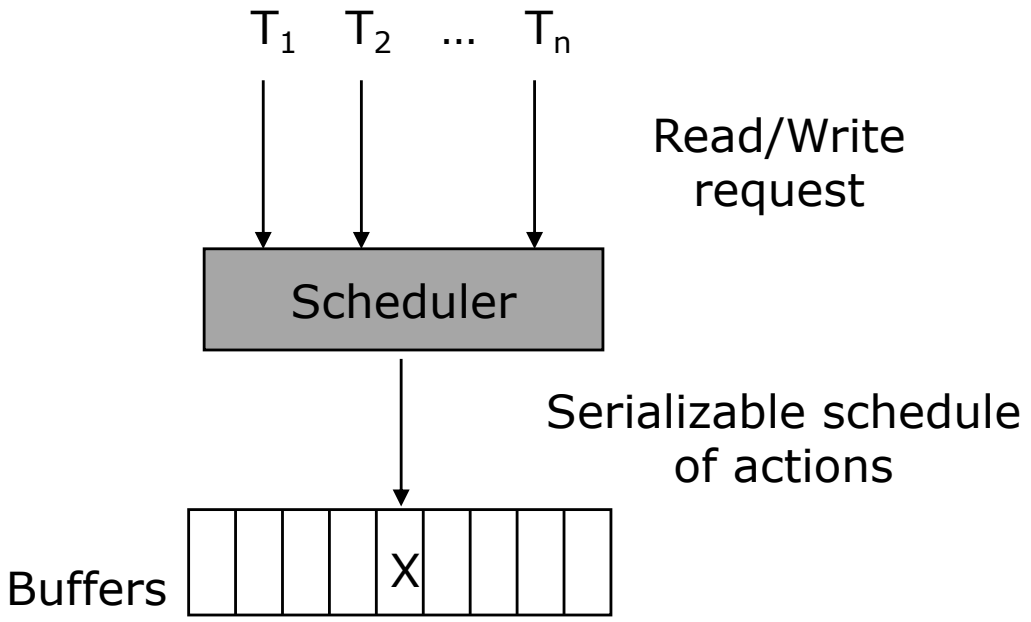
- In reality
  - What will commercial DBMSs do?

Content

- Issues in concurrency control
- **Lock protocol**
  - Introduction
  - Two-phase locking
  - Lock modes
  - Multiple granuraity
  - Tree-base
- More on transaction processing

Remind

- Responsibility of scheduler
  - Take requests from transactions
  - Either allow requests to operate on the database
  - Or defer requests for a while
    - Until it is safe to allow requests to execute



Discussion

- When are these requests operated?
- When are these requests deferred?
- How to prevent a cyclic P(S) at the time of execution?

Idea

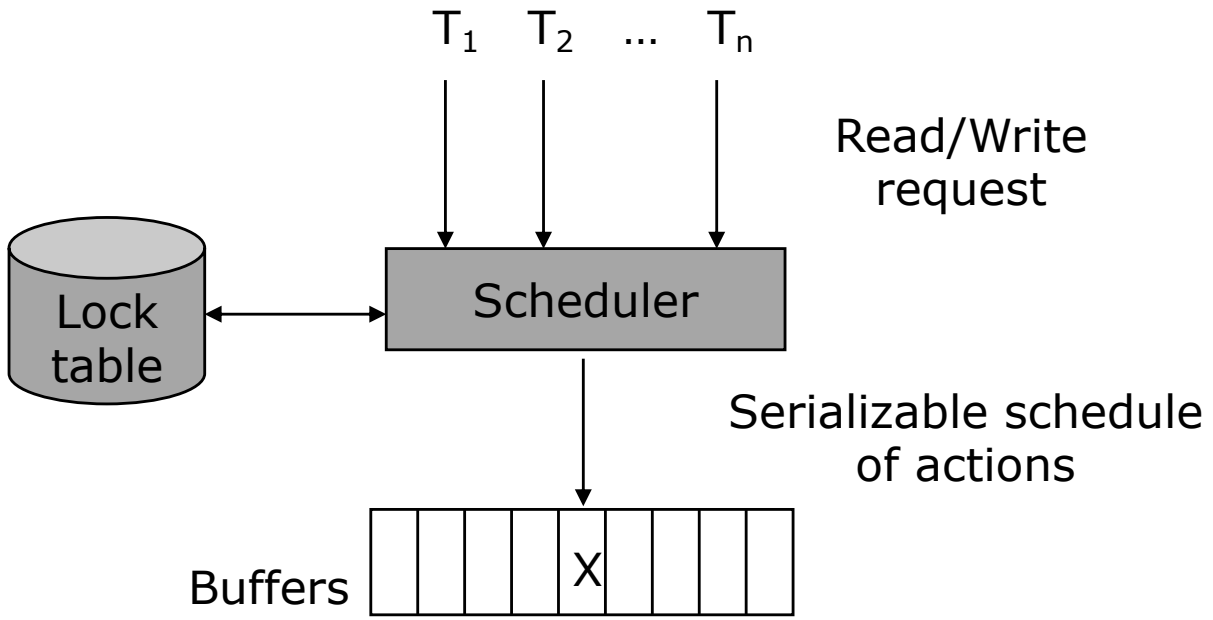
- Instead enforce conflict-serializabliity, schedulers use a simple test
  - Guarantee serializability
  - May forbid some actions that could lead to inconsistency

→ ***Locking*** scheduler

- Note
  - More stringent condition than serializability

Locking scheduler

- Add two more actions
  - Lock action
  - Unlock action



Locking scheduler

- Transactions must request a ***lock*** on a database element X before read/write
  - Lock(X) or l(X)
- This request is managed by ***lock manager***
  - If it is accepted, transactions will be allowed to read/write on database elements
- After read/write, transactions must release (***unlock***) the database elements
  - Unlock(X) or u(X)

Locking scheduler

- The use of lock must be proper in two senses
  - *(1) Consistency of transactions*

$$T_i : \quad \dots\ l(X)\ \dots\ r(X)/w(X)\ \dots\ u(X)\ \dots$$

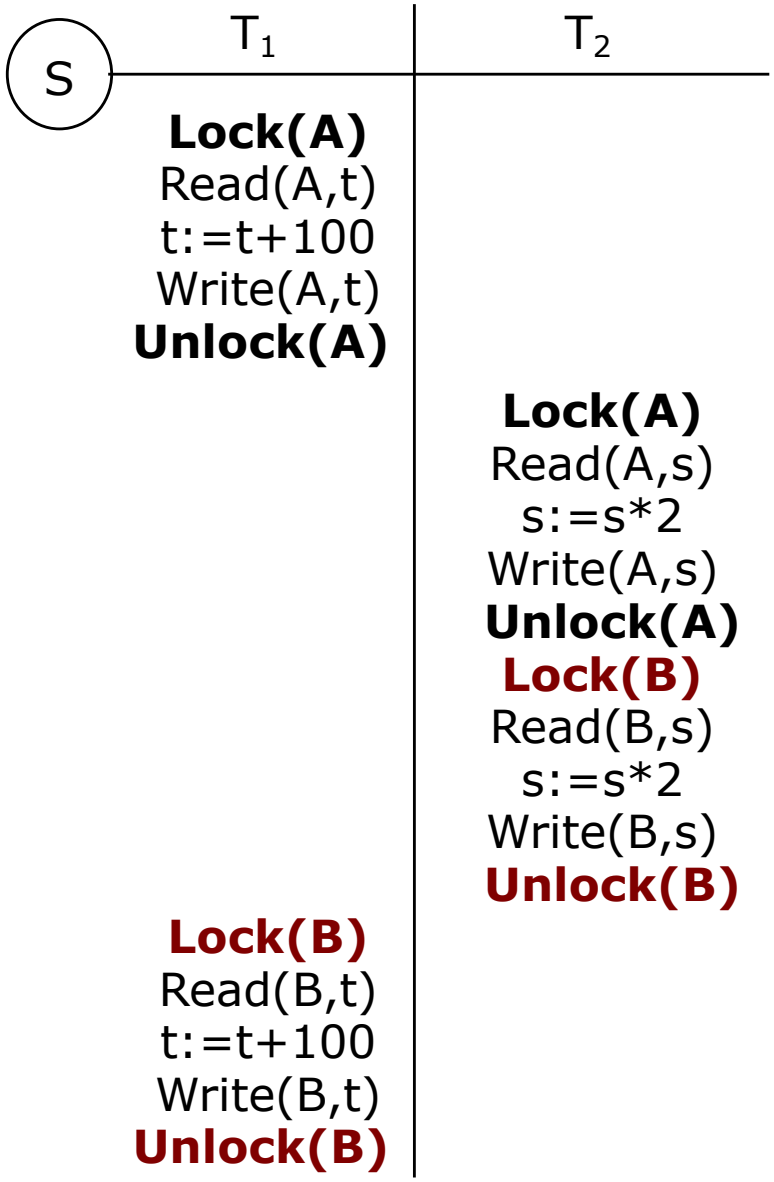
  - *(2) Legality of schedules*

$$S : \quad \dots\ l_i(X)\ \dots\dots\dots\dots\dots\ u_i(X)\ \dots$$

$$\longleftrightarrow$$

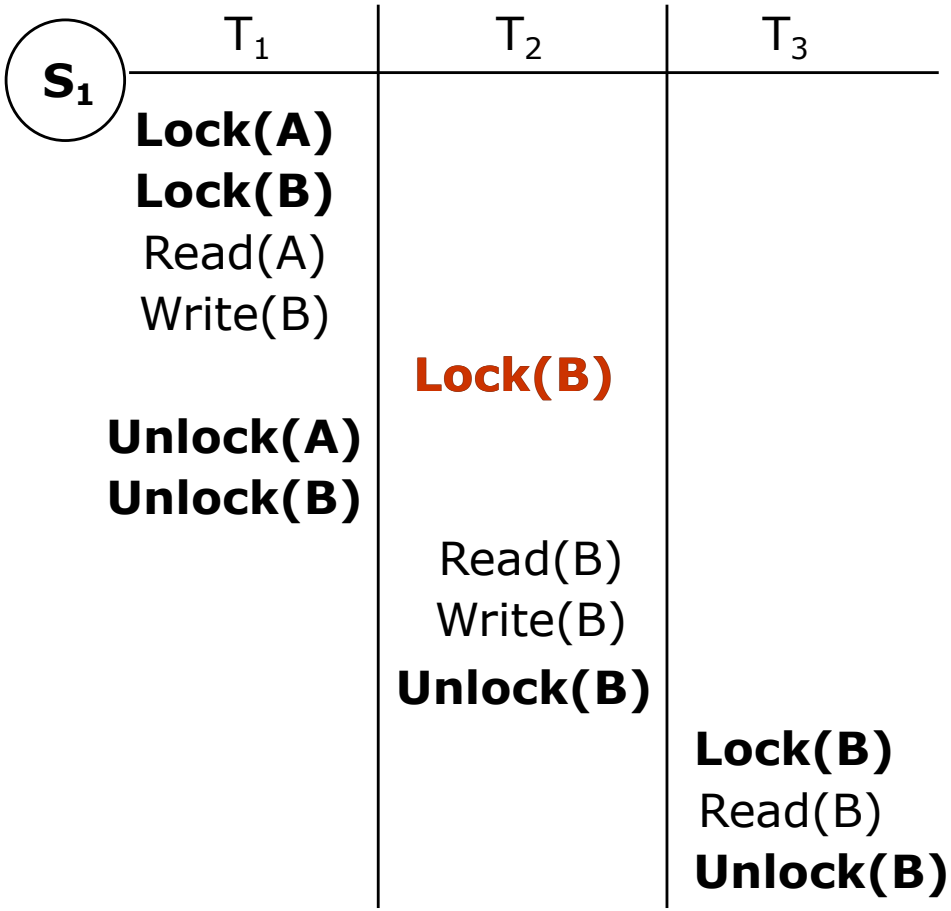
$$\text{no } l_j(X)$$

Example


S
T1
T2
Lock(A)
Read(A,t)
t:=t+100
Write(A,t)
Unlock(A)
Lock(A)
Read(A,s)
s:=s*2
Write(A,s)
Unlock(A)
Lock(B)
Read(B,s)
s:=s*2
Write(B,s)
Unlock(B)
Lock(B)
Read(B,t)
t:=t+100
Write(B,t)
Unlock(B)

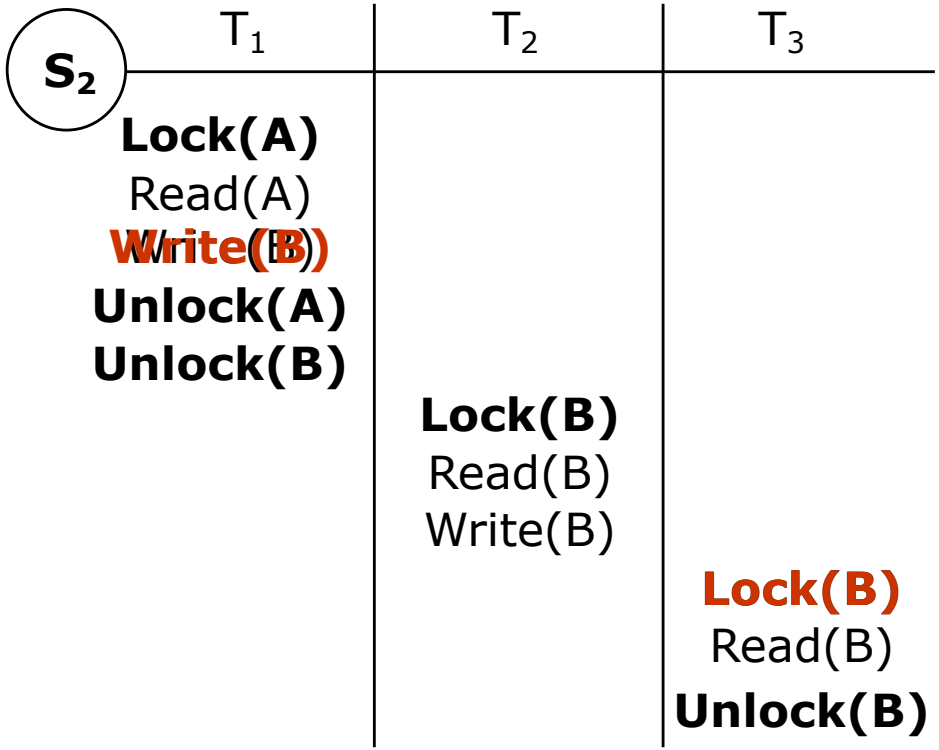
Exercise

- What are consistent transactions?
- Is the schedule legal?



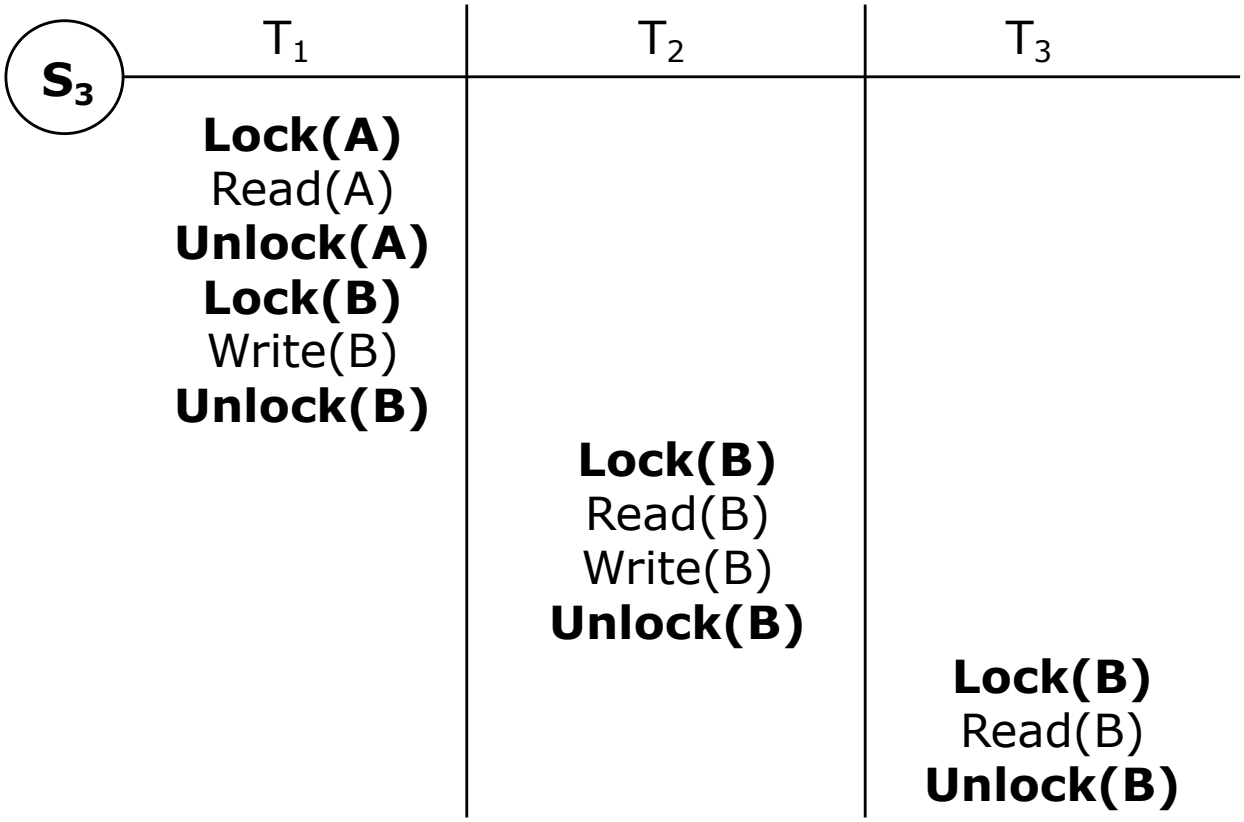
Exercise

- What are consistent transactions?
- Is the schedule legal?



Exercise

- What are consistent transactions?
- Is the schedule legal?



Discussion

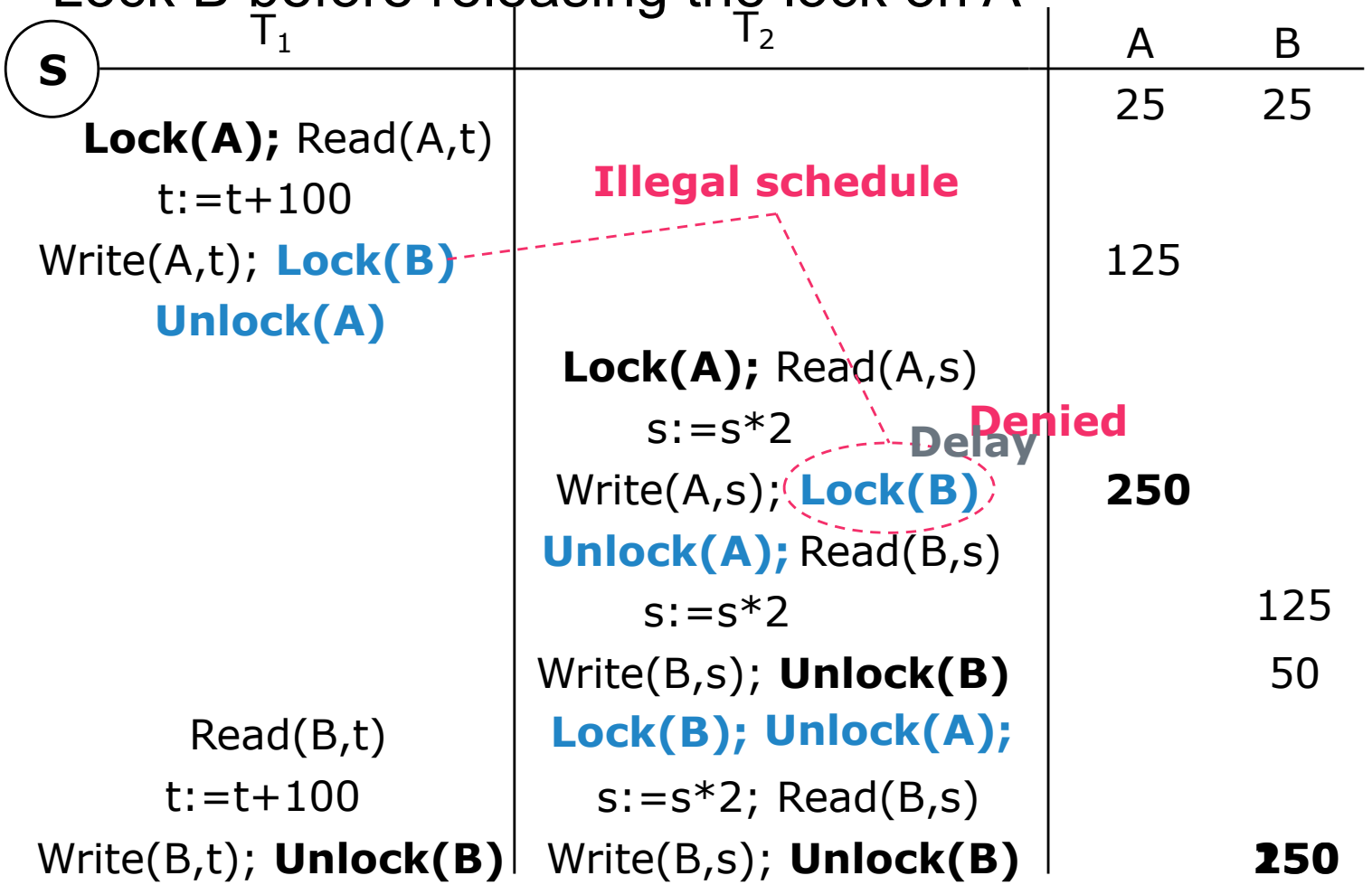
- Suppose S is legal, is S serializable?

S	T_1	T_2	A	B
			25	25
	Lock(A); Read(A,t)			
	t:=t+100			
	Write(A,t); **Unlock(A)**		125	
		Lock(A); Read(A,s)		
		s:=s*2		
		Write(A,s); **Unlock(A)**	**250**	
		Lock(B); Read(B,s)		
		s:=s*2		
		Write(B,s); **Unlock(B)**		50
	Lock(B); Read(B,t)			
	t:=t+100			
	Write(B,t); **Unlock(B)**			**150**

Not serializable

Discussion

- A simple change
  - Lock B before releasing the lock on A

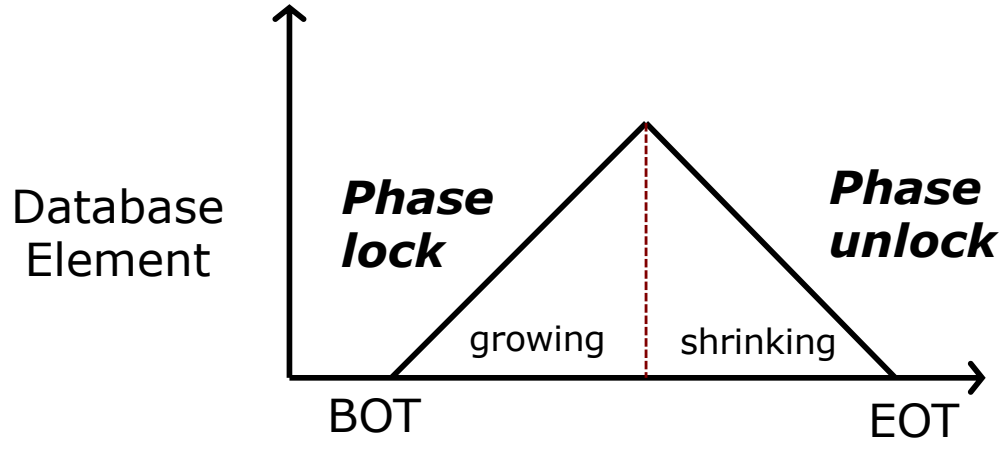


Two-phase locking

- A condition on the order of actions in a transaction
  - *(3) Two-phase locked transactions*

T_i : ..... $l_i(X)$ ..... $u_i(X)$ ...

no unlock **no lock**



Example

T_1
Lock(A)
Read(A)
Lock(B)
Read(B)
B:=B+A
Write(B)
Unlock(A)
Unlock(B)

T_2
Lock(B)
Read(B)
Lock(A)
Read(A)
Unlock(B)
A:=A+B
Write(A)
Unlock(A)

2PL transactions

T_3
Lock(B)
Read(B)
B=B-50
Write(B)
Unlock(B)
Lock(A)
Read(A)
A=A+50
Write(A)
Unlock(A)

T_4
Lock(A)
Read(A)
Unlock(A)
Lock(B)
Read(B)
Unlock(B)
Pritn(A+B)

Non-2PL transactions

Theorem

- A schedule S satisfies
  - *(1) Consistency of transactions*
  - *(2) Legality of schedules*
  - *(3) Two-phase locked transactions*
- $\Longrightarrow$ S will be converted to a conflict-equivalent serial schedule
  - S conflict serializable schedule

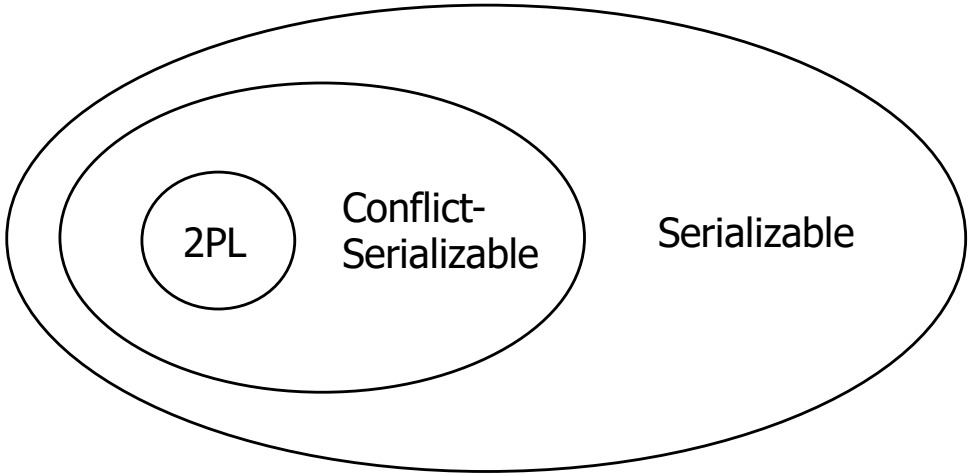
- Note
  - Non 2PL transactions allow not-conflict-serializable schedule

Proof

- Suppose P(S) is cyclic
  - $T_1 \rightarrow T_2 \rightarrow \ldots \rightarrow T_n \rightarrow T_1$
- T_1 executes locks on elements unlocked by T_n
  - T_1 has a sequence of actions as follows
    - Lock … Unlock … Lock
- This is illegal
  - Because all transactions in S are two-phase locked transactions
- P(S) is acyclic

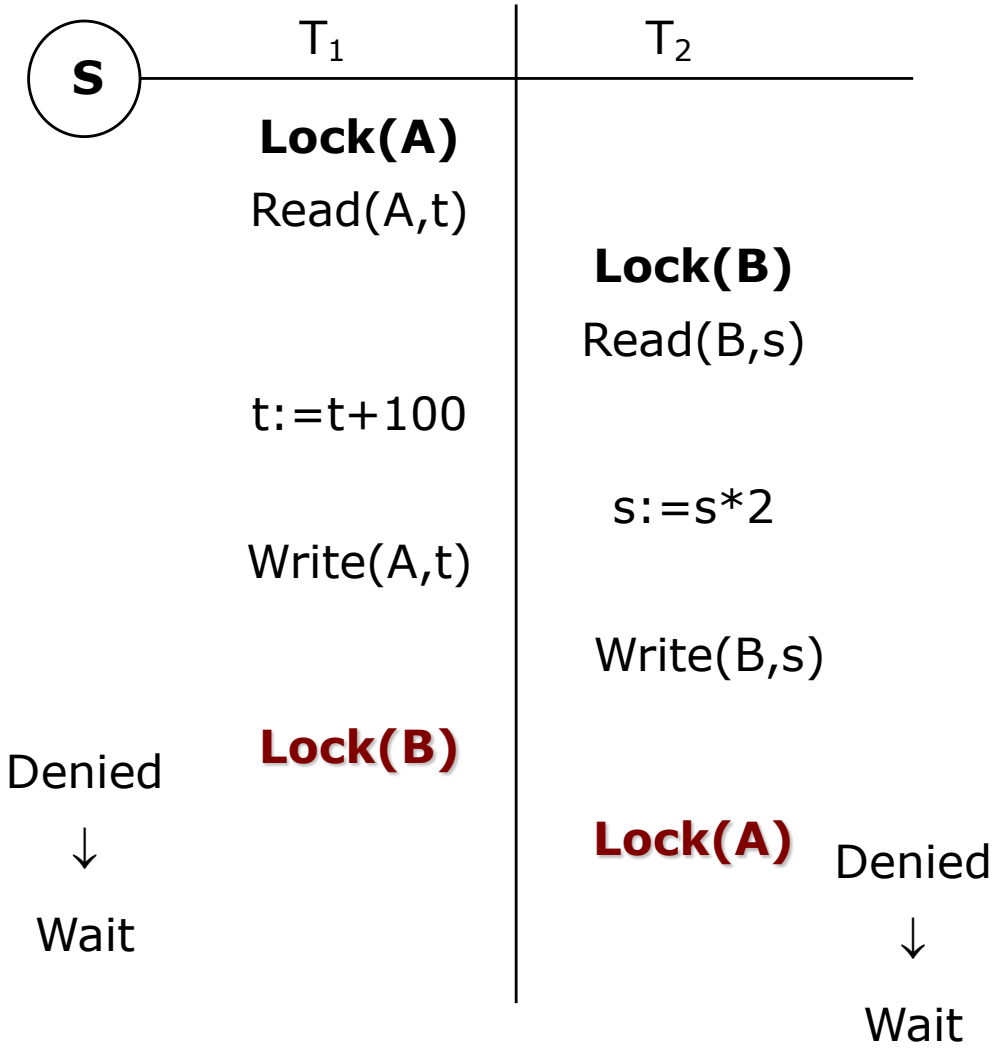
Discussion

- Serializable
- Conflict-serializable
- 2PL



- S: w1(X) w3(X) w2(Y) w1(Y)
  - S is conflict-serializable ???
  - S satifies 2PL ???
    - w1(X) *u1(X)* **w3(X)** w2(Y) ***l1(Y)*** ***u1(X)*** w1(Y)

Discussion



Deadlock problem

is not solved by

two-phase locking

Beyond simple lock protocol

- Improving performance
- Allowing more concurrency

- Lock mode
  - Read/Write lock
  - Increment lock
  - Update lock
- Multiple granularity
- Tree-base

Problem

- A transaction T must lock on element X, even if it only wants to read X and not write it
- But, we cannot avoid taking the lock
  - If not, unserializable behavior will happens
- Why many transactions could not read X at the same time, as long as none is allowed to write X?

T_i	T_j
Lock(X) Read(X) Unlock(X)	
	Lock(X) Read(X) Unlock(X)

Lock for writing is "stronger" than lock for reading

Lock mode

- Locking scheduler uses 2 different kinds of locks
  - Shared lock
    - Read lock
    - rl(X) or RLock(X)
  - Exclusive lock
    - Write lock
    - wl(X) or WLock(X)
  - Release
    - Unlock
    - u(X) or Unlock(X)

Lock mode

- For any database element X
  - Either one exclusive lock on X
  - Or no exclusive locks but any number of shared locks

  - If write(X)
    - Need to have WLock(X)
  - If read(X)
    - May have either RLock(X) or WLock(X)

Lock mode

- Three kinds of requirements
  - (1) Consistency of transactions

T_i : ... rl(X)/wl(X) ... r(X) ... u(X) ...

T_i : ... wl(X) ... w(X) ... u(X) ...

Lock mode

- Three kinds of requirements
  - (2) Legality of schedules

S : … $rl_i(X)$ ……………… $u_i(X)$ …

$\longleftrightarrow$

no $wl_j(X)$

S : … $wl_i(A)$ ……………… $u_i(A)$ …

$\longleftrightarrow$

no $wl_j(A)$

no $rl_j(A)$

Lock mode

- Three kinds of requirements
  - (3) Two phase locking of transactions

T : ... rl(X)/wl(X) ..... u(X) ...

no unlock **no lock**

Discussion

- Allow one transaction to request and hold both shared and exclusive lock on the same element

  - If transactions know in advance their needs for locks
    - Only exclusive lock

      T_i : … wl(A) … r(A) … w(A) … u(A) …

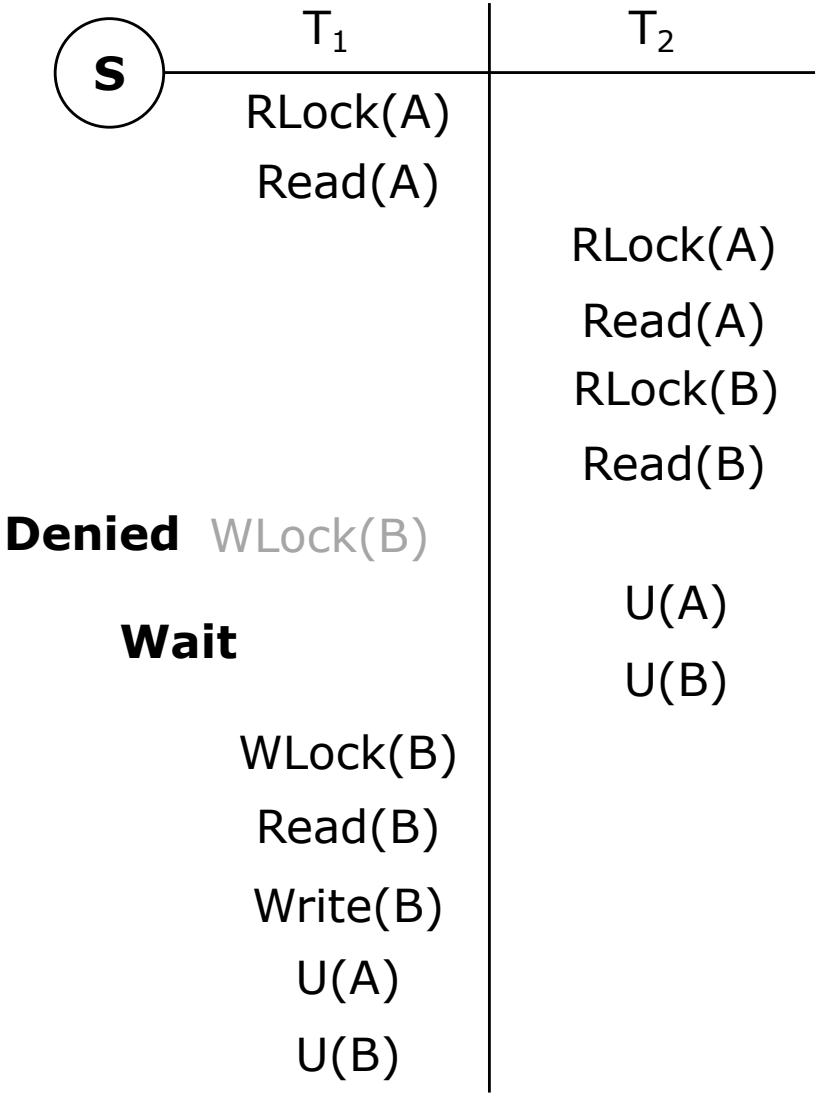
  - If lock needs are unpredictable
    - Both shared and exclusive locks at different times

      T_i : … rl(A) … r(A) … wl(A) … w(A) … u(A) …

      - get the 2nd lock on A
      - drop read lock, get write lock

T_1
Read(A)
Write(A)?

Example



- S is conflict-serializable
- The order is (T_2,T_1)
  - Even though T_1 started first
  - But T_2 unlocks before T_1

Theorem

- In systems with shared and exclusive locks,
  - S must satisfy (1) & (2) & (3)
  - $\Longrightarrow$ S is conflict-serializable

- Note
  - Non 2PL transactions allow not-conflict-serializable schedule

Compatibility matrix

- Using several lock modes
  - Scheduler needs a policy
    - When it can grant a lock request, given other locks that may already be held on the same database element

		Lock requested	
		Share	**eXclusive**
Lock held in mode	**Share**	**yes**	**no**
	eXclusive	**no**	**no**

Upgrading locks

- Examine a case
  - Transaction T take a ***shared lock*** on X
  - T wants to write a new value of X
  - When T is ready to write, ***upgrade the lock to exclusive***
    - Request an exclusive lock on X in addition to its already held shared lock on X
  - Nothing prevents a transaction from issuing requests for locks on the same DB element in different modes

Upgrading locks

- Examine a case

T_1
Read(A)
Write(A)?

- Solution
  - (1) Request exclusive lock

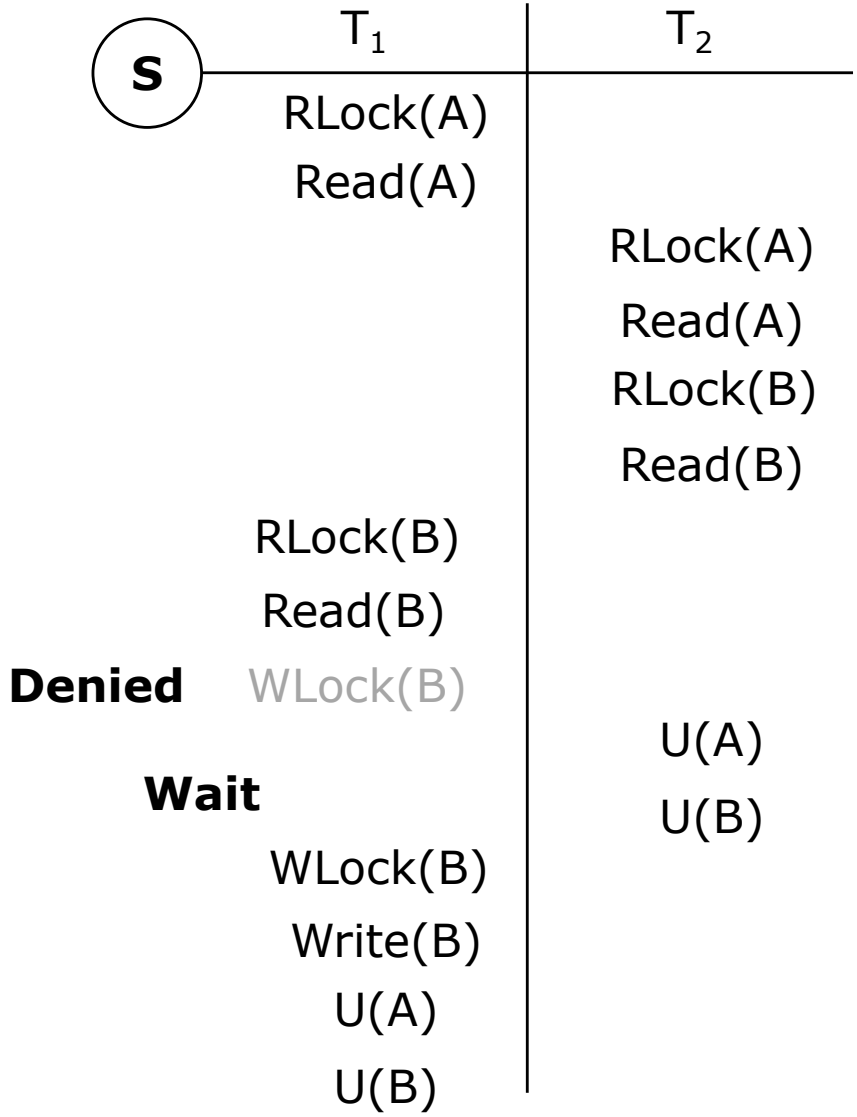
T_i : ... wl(A) ... r(A) ... w(A) ... u(A) ...

  - (2) Upgrade exclusive lock

T_i : ... rl(A) ... r(A) u(A) wl(A) ... w(A) ... u(A)

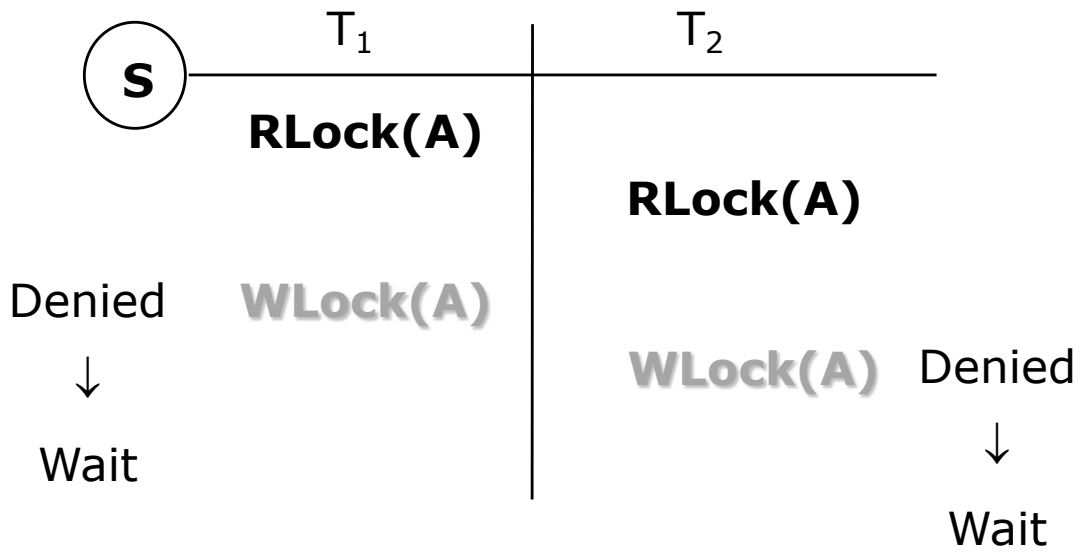
can be allowed in growing phase

Upgrading locks



- T_2 gets a shared lock on B before T_1 does
- T_1 is also able to lock B in shared mode
- T_1 tried to upgrade its lock on B to exclusive

Discussion



Upgrading by two transactions can cause a deadlock

Update locks

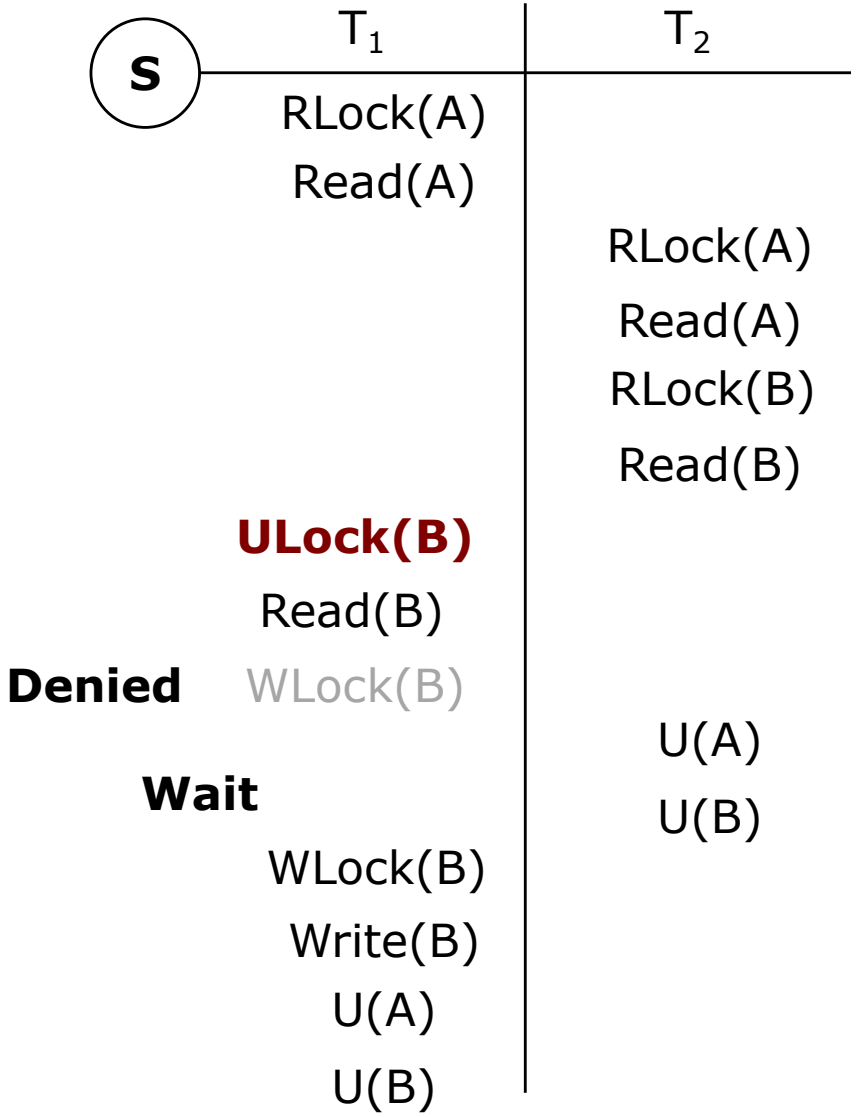
- Avoiding the deadlock
  - A third lock mode, ***update lock***
    - ul(X) or ULock(X)
    - Give the transaction T only the privilege to **read X, but not to write X**
    - Only update lock **can be upgraded to a write lock**, a read lock cannot be upgraded
    - Grant update lock on X when X was already locked by shared locks
    - Then, we prevent additional locks of any kind – shared, update, exclusive – on X

Update locks

- Compatibility matrix

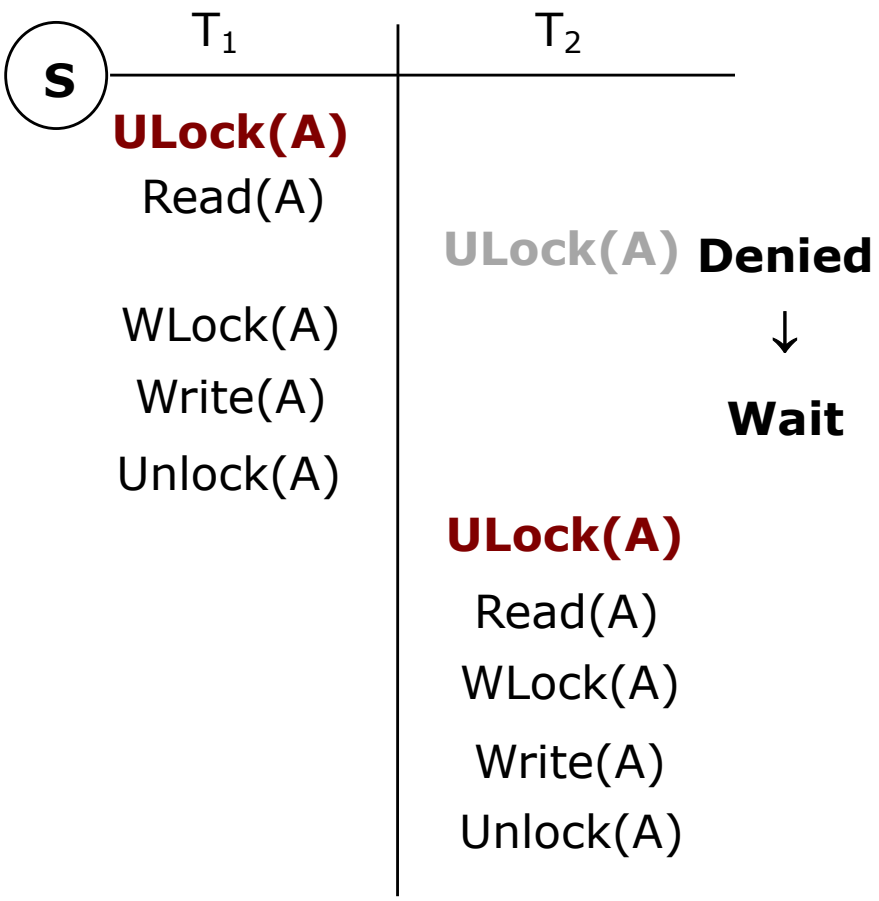
		Lock requested		
		Share	**eXclusive**	**Update**
Lock held in mode	**Share**	**yes**	**no**	**yes**
	eXclusive	**no**	**no**	**no**
	Update	**no**	**no**	**no**

Example



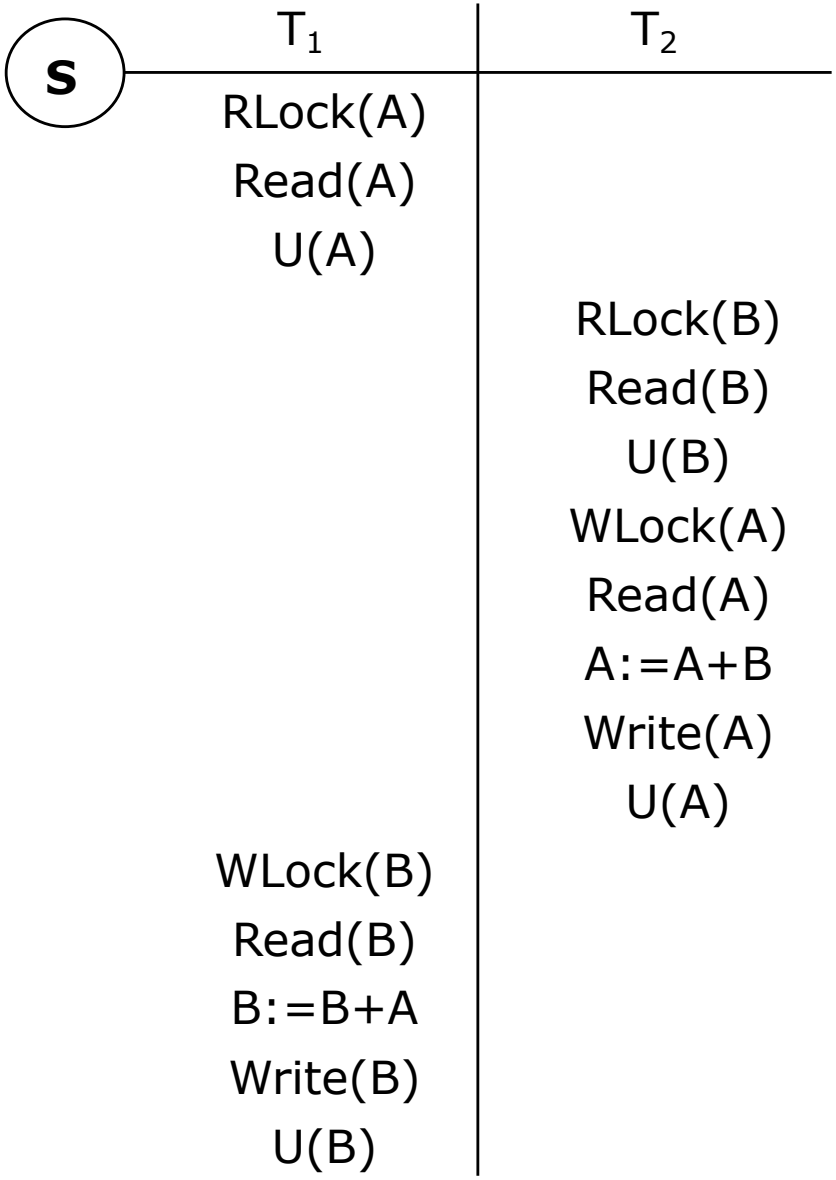
- T_1 would take an update lock on B, rather than a shared lock

Example



- Update locks fix the deadlock problem

Exercise



- Which transactions are not 2PL?
- Is S serializable?

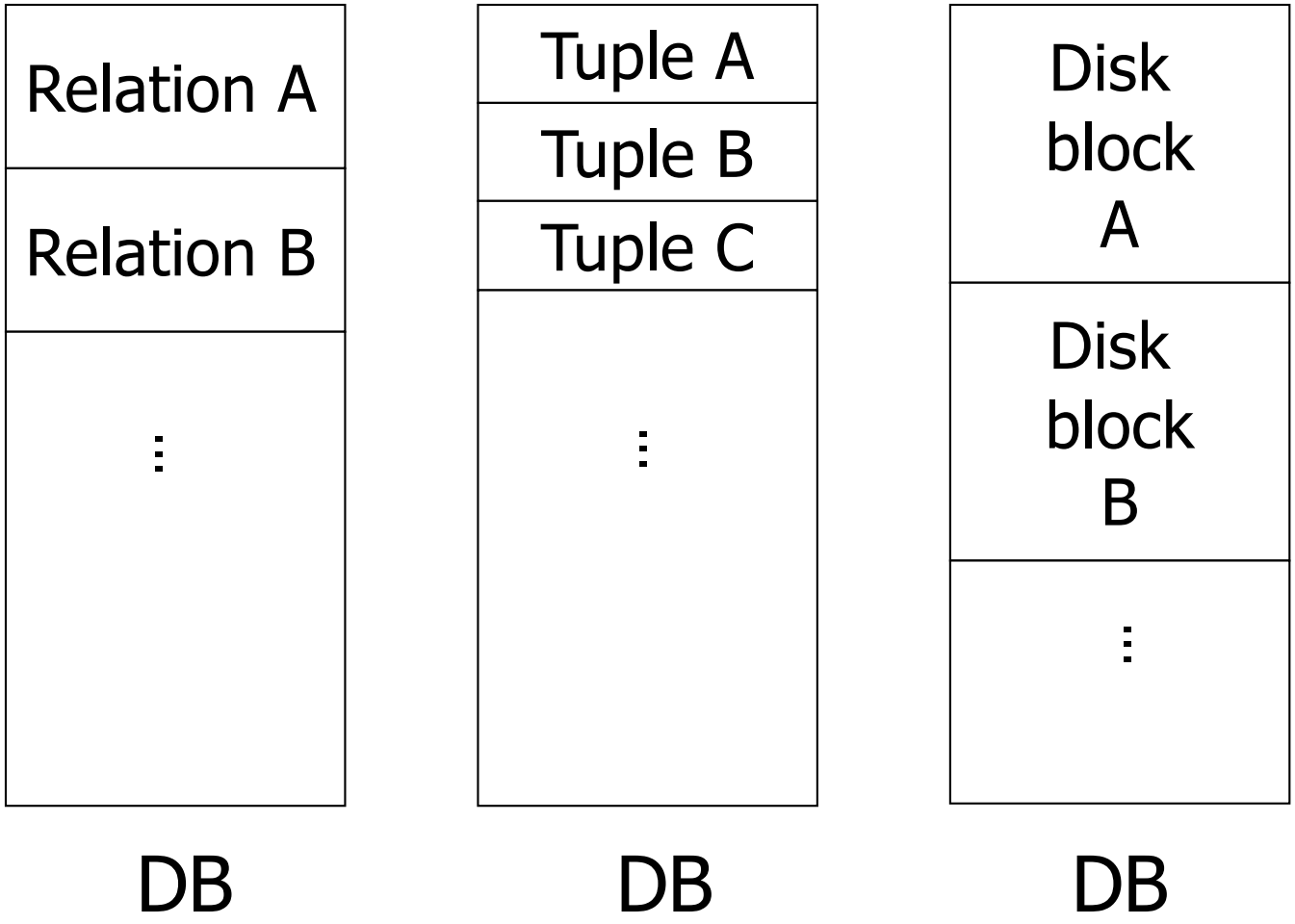
Exercise

s

T_1	T_2	T_3	T_4
	RL(A)		
		RL(A)	
	WL(B)		
	U(A)		
		WL(A)	
	U(B)		
RL(B)			
		U(A)	
			RL(B)
RL(A)			
			U(B)
WL(C)			
U(A)			
			WL(B)
			U(B)
U(B)			
U(C)			

- Which transactions are not 2PL?
- Is S serializable?

Which data units do we lock?

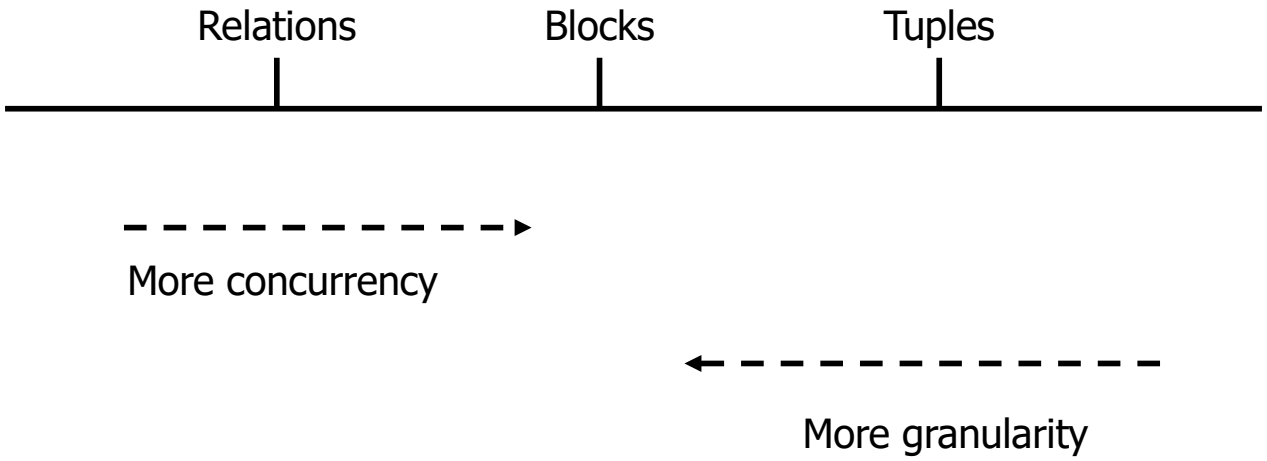


Banking example

- Relation $account(accID, balance)$
- Withdraw and deposit
  - Lock relation
    - A transaction changes the balance's value : WLock request
    - Only withdraw or deposit at a time
    - → Low concurrency
  - Lock tuples or disk blocks
    - If 2 accounts are in 2 different blocks : update balances at a time
    - → More concurrency
- Sum of balances
  - Lock relation ???
  - Lock tuple or disk block ???

Multiple granularity locking

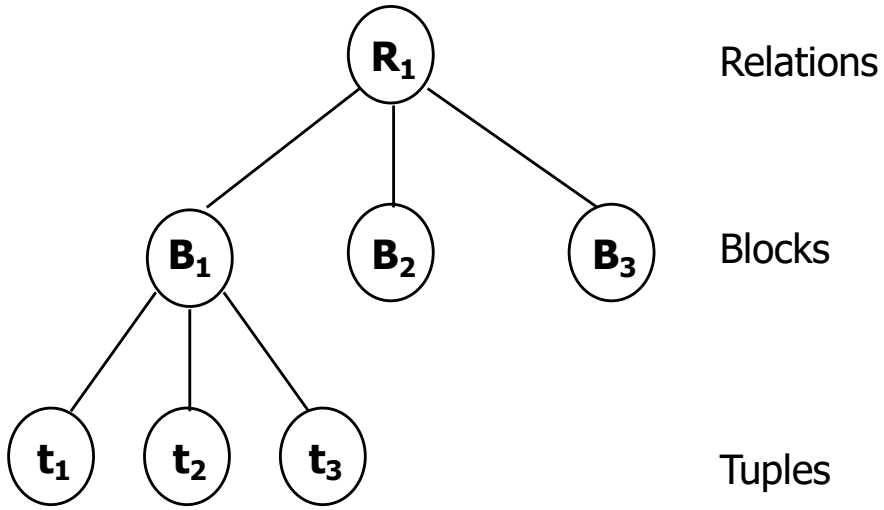
- Manage locks at different granularities



We can have both ways ???

DB element hierarchy

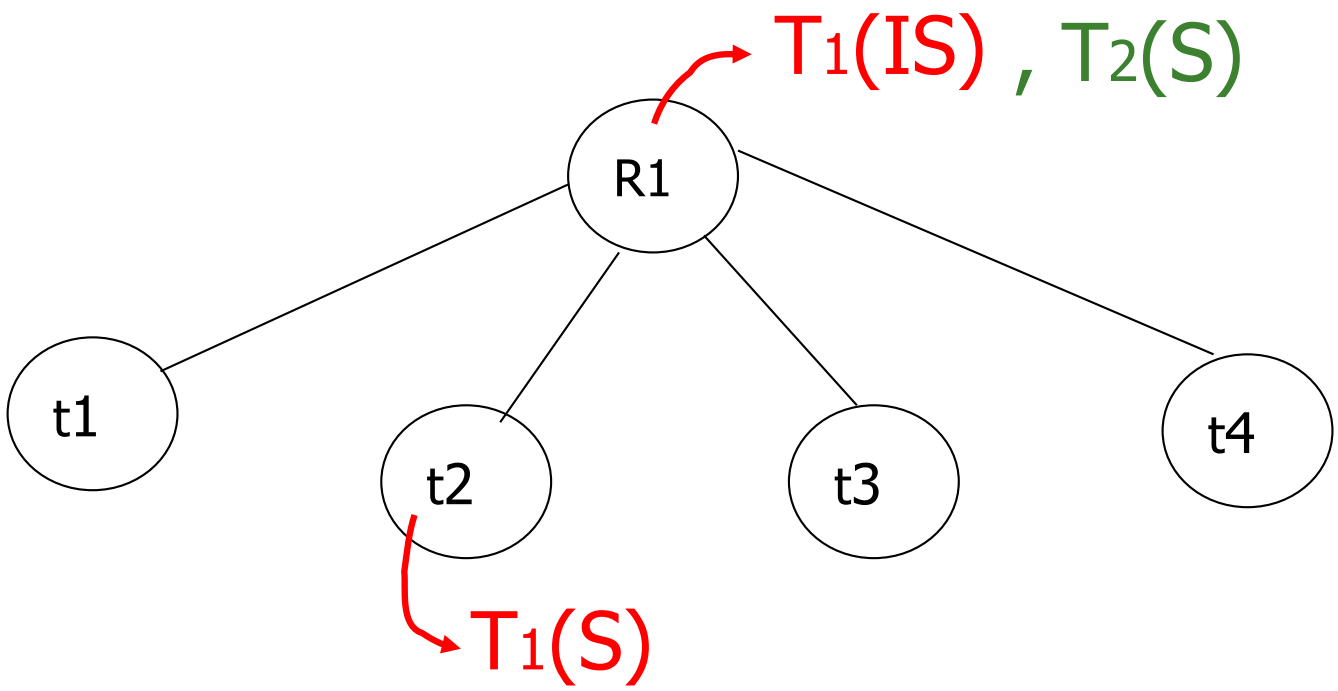
- Relations are the largest lockable elements
- Each relation is composed of 1 or more blocks
- Each block contains 1 or more tuples



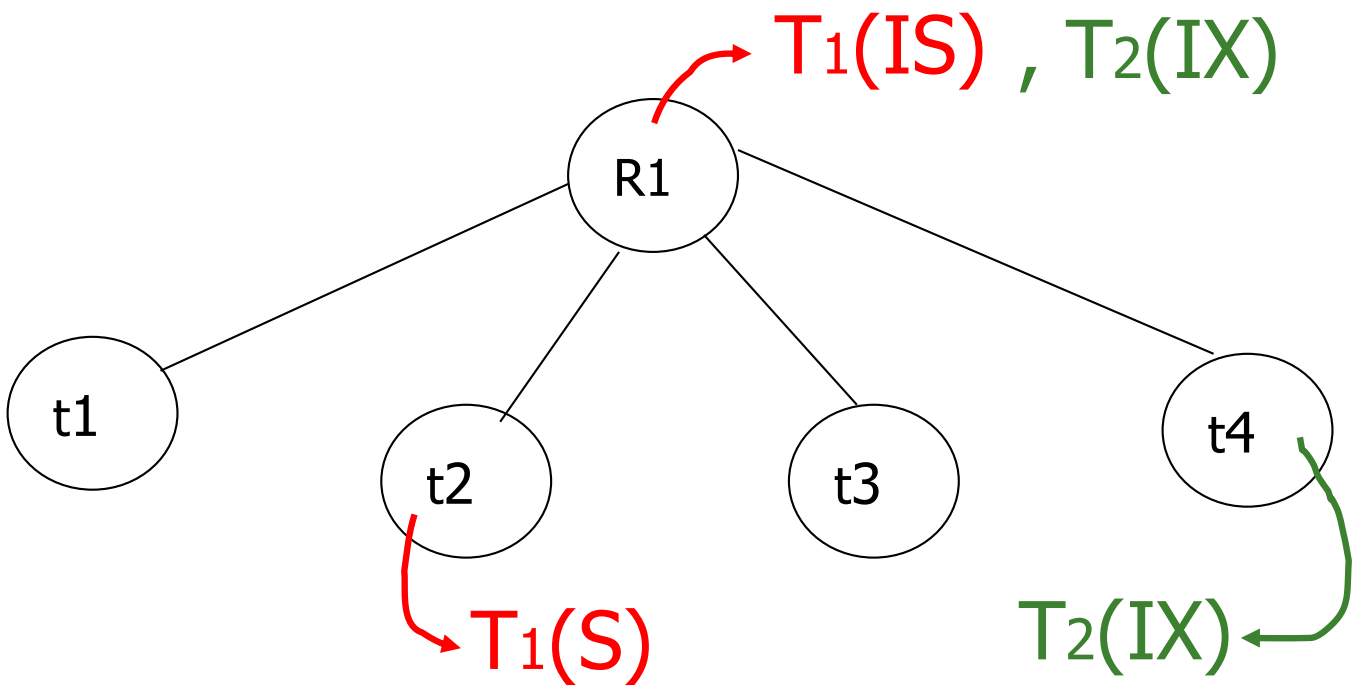
Multiple granularity locking

- Involves both
  - Ordinary locks
    - Shared lock : S
    - Exclusive lock : X
  - Warning clocks
    - Warning (intention to) shared lock : IS
    - Warning (intention to) exclusive lock : IX

Example



Example



Multiple granularity locking

- Compatibility matrix

Holder \ Requestor	IS	IX	S	SIX	X
IS	**yes**	**yes**	**yes**	**yes**	**no**
IX	**yes**	**yes**	**no**	**no**	**no**
S	**yes**	**no**	**yes**	**no**	**no**
SIX	**yes**	**no**	**no**	**no**	**no**
X	**no**	**no**	**no**	**no**	**no**

Multiple granularity locking

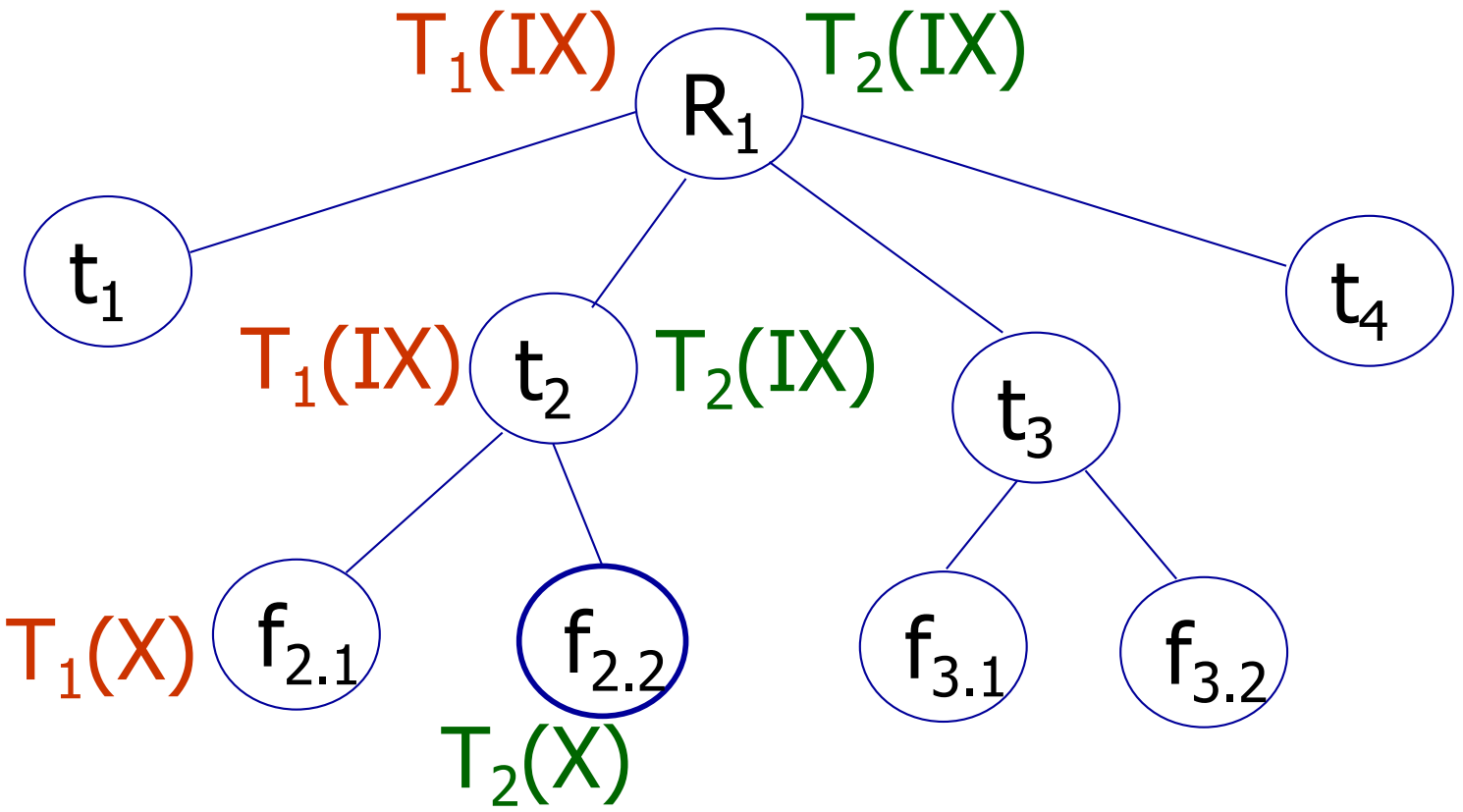
- Parent-child rule

Parent locked in	Child can be locked in by the same transaction
IS	**IS, S**
IX	**IS, S, IX, X**
S	**[S, IS] not necessary**
SIX	**X, IX, [SIX]**
X	**none**

Rules

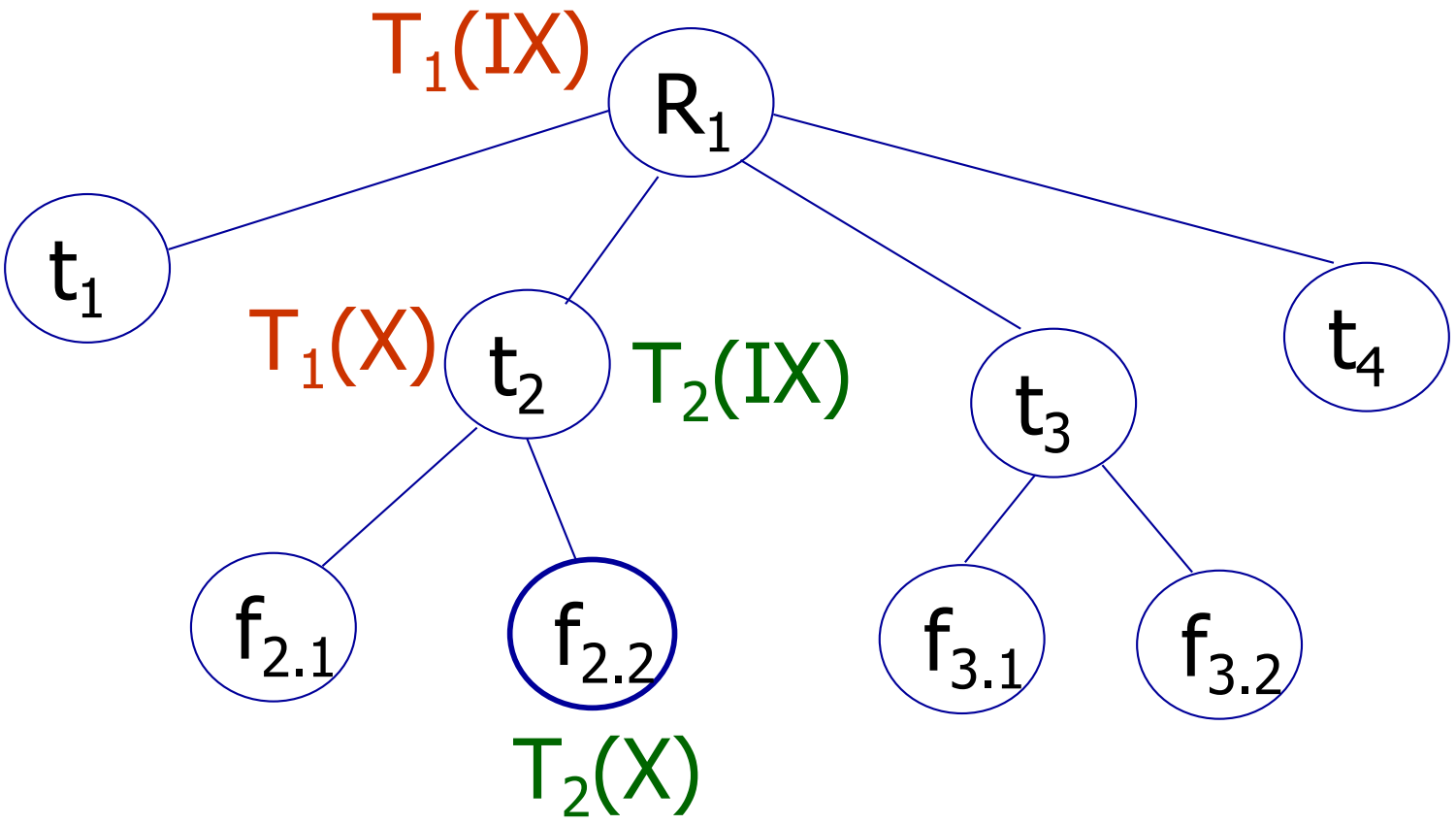
- (1) Follow multiple granularity compatibility matrix
- (2) Lock root of tree first, any mode
- (3) Node Q can be locked by Ti in S or IS only if parent(Q) locked by Ti in IX or IS
- (4) Node Q can be locked by Ti in X,SIX,IX only if parent(Q) locked by Ti in IX,SIX
- (5) Ti is two-phase
- (6) Ti can unlock node Q only if none of Q's children are locked by Ti

Exercise



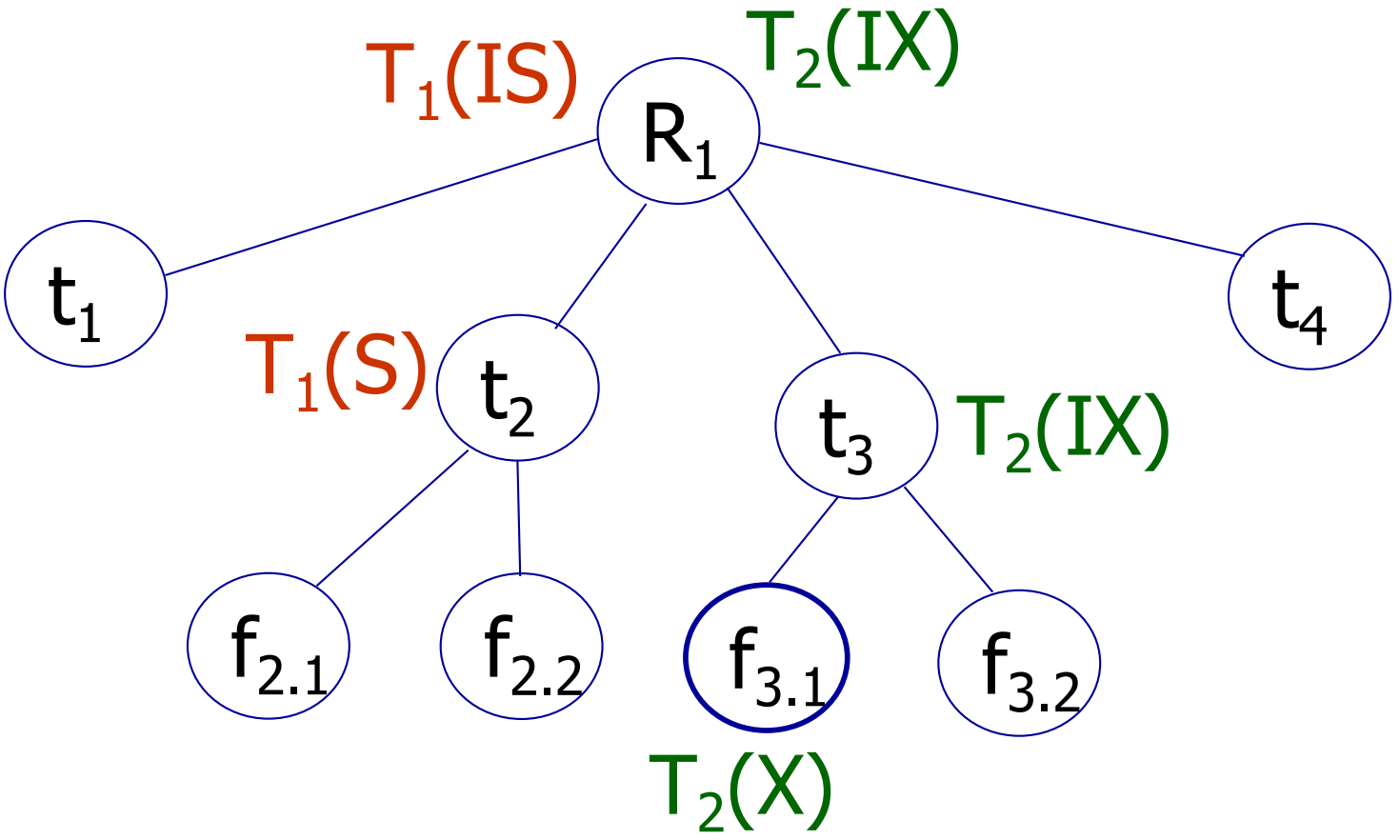
- Can T2 access object f2.2 in X mode?
- What locks will T2 get?

Exercise



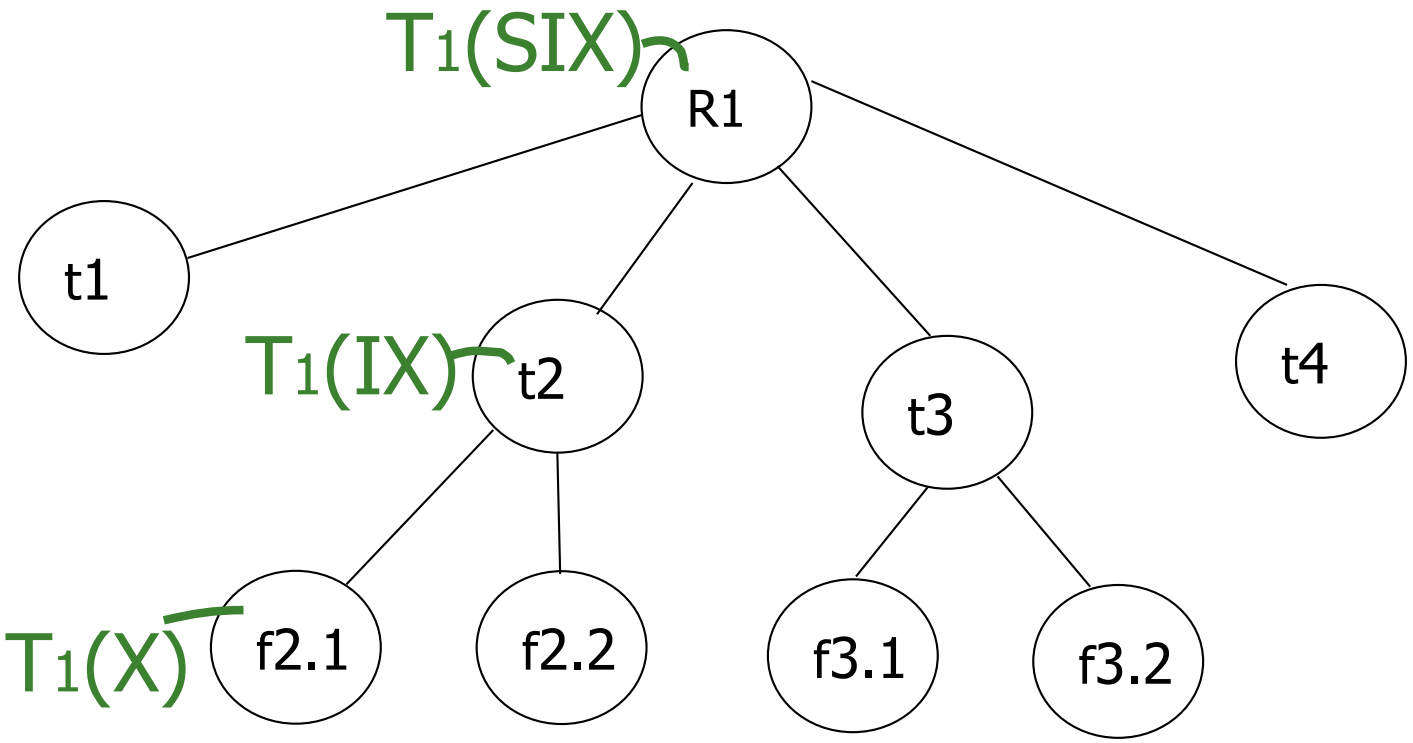
- Can T2 access object f2.2 in X mode?
- What locks will T2 get?

Exercise



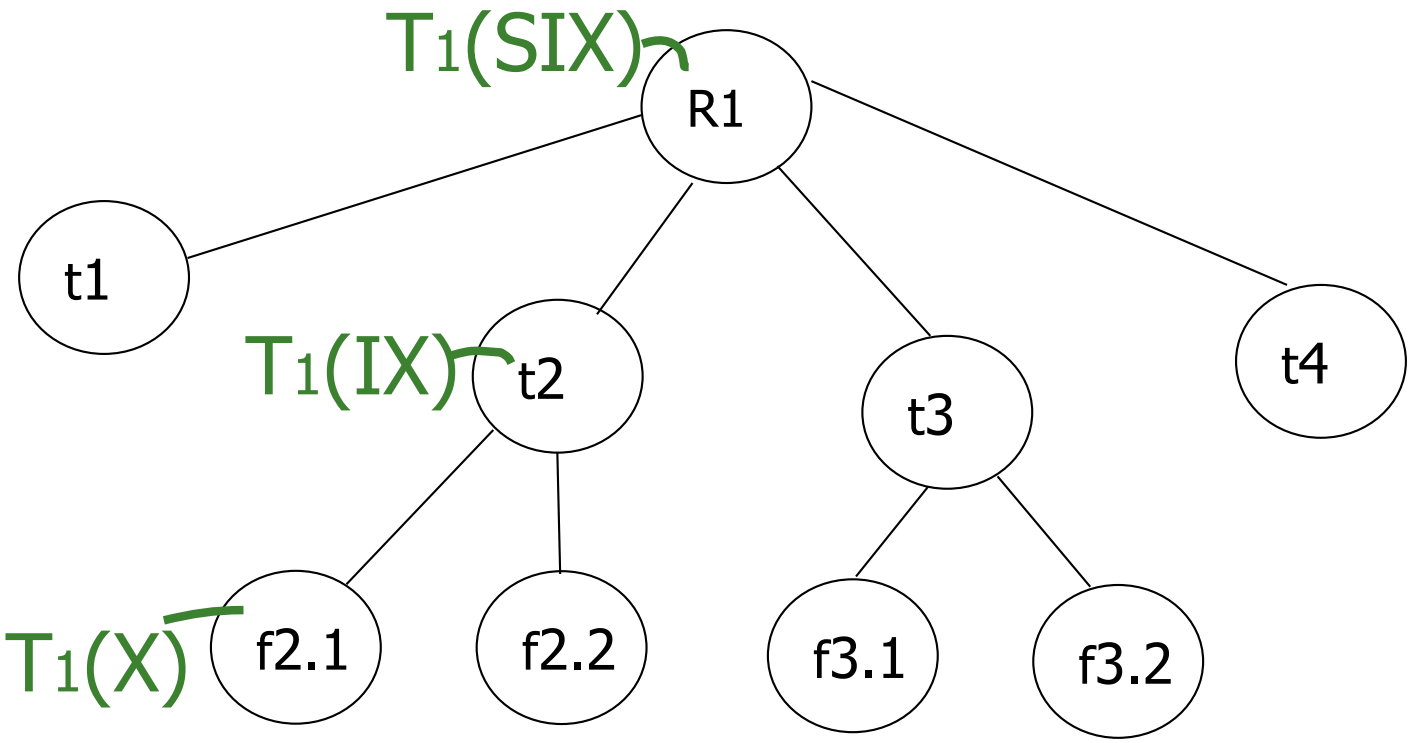
- Can T_2 access object $f_{3.1}$ in X mode?
- What locks will T_2 get?

Exercise



- Can T_2 access object $f_{2.2}$ in S mode?
- What locks will T_2 get?

Exercise

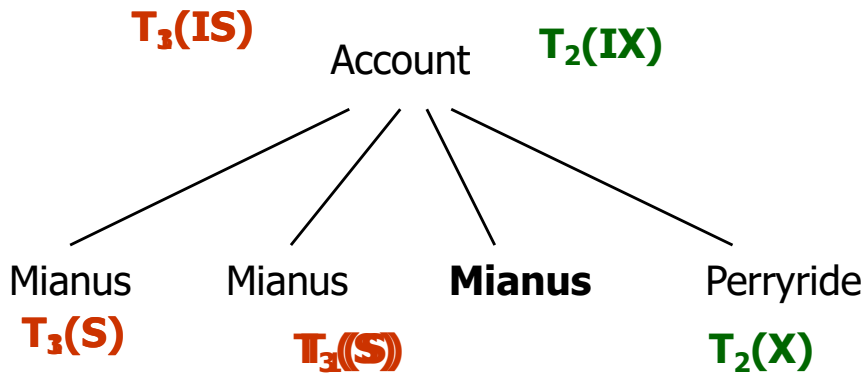


- Can T_2 access object $f_{2.2}$ in X mode?
- What locks will T_2 get?

Discussion

Account

accID	balance	branchID
A-101	500	Mianus
A-215	700	Mianus
A-102	400	Perryride
A-201	900	Mianus



T_3 is still correct ???

Before insert of node Q, lock parent(Q) in X mode

T_1

```
select *  from account
where branchID="Mianus"
```

T_2

```
update account
set balance=400
where branchID="Perryride"
```

T_3

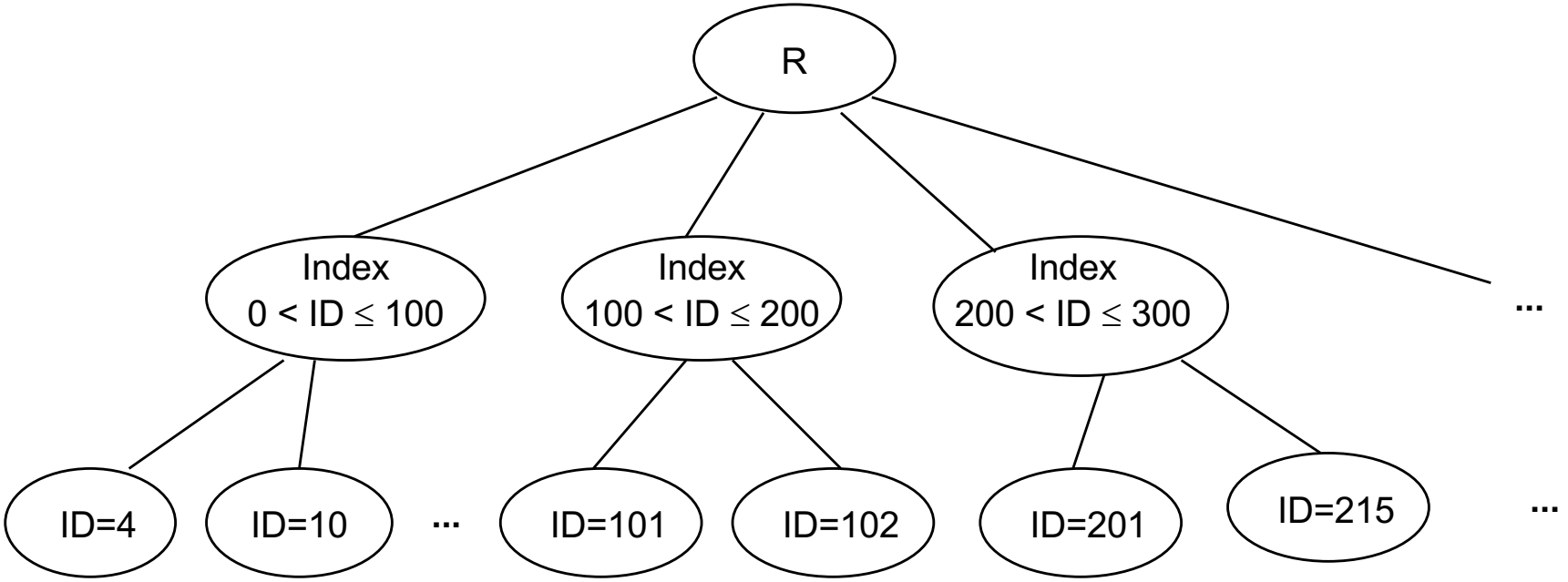
```
select sum(balance)
from account
where branchID="Mianus"
```

T_4

```
insert account
valuse("A-201", 900, "Mianus")
```

Another approach

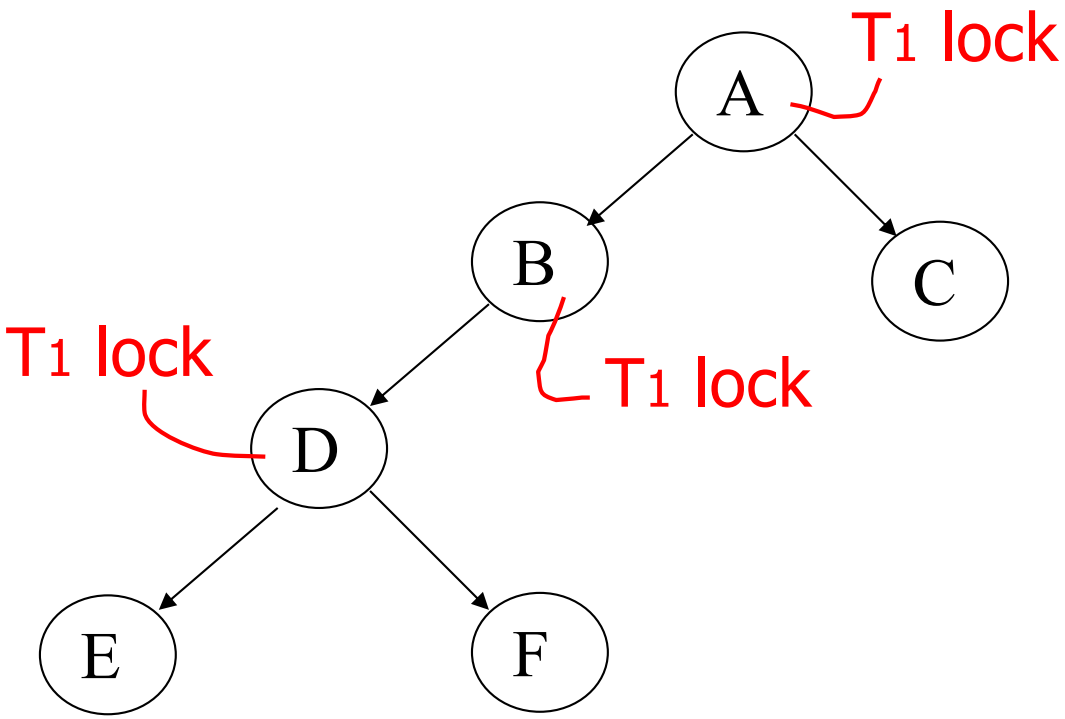
- Instead of using R, can use index on R



- It can be generalized to multiple indexes

Tree protocol

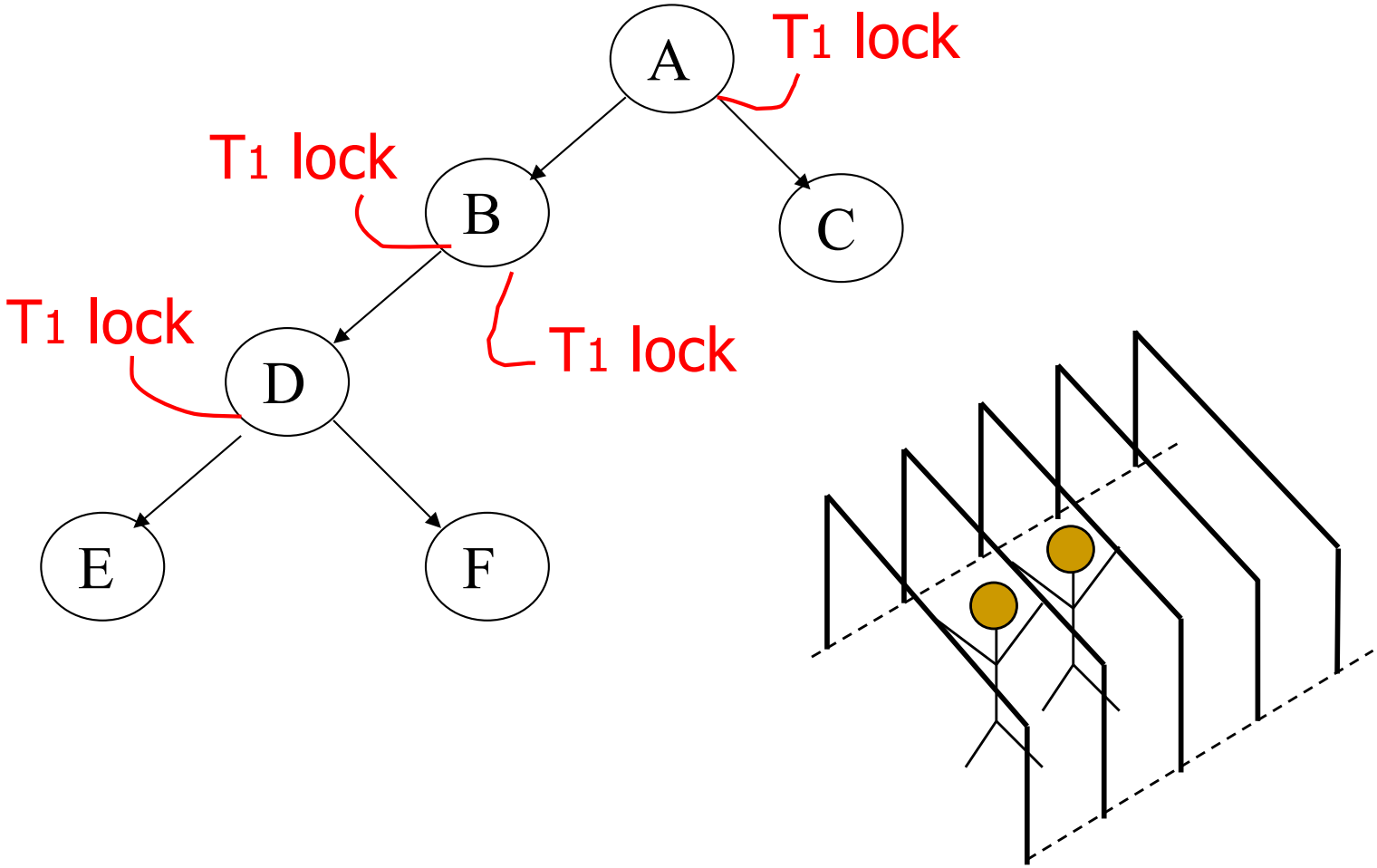
- All objects accessed through root, following pointers



- Can we release A lock if we no longer need A?

Idea

- Traverse like “Monkey Bars”



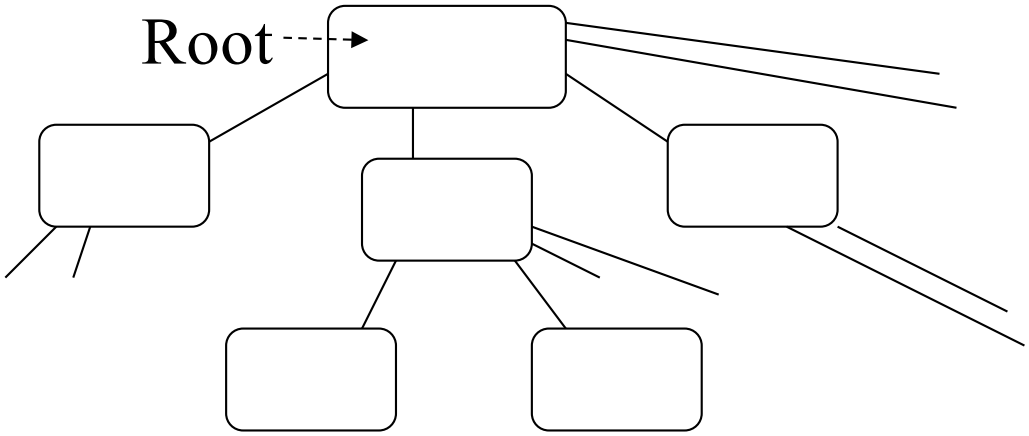
Rules

- (1) First lock by Ti may be on any item
- (2) After that, item Q can be locked by Ti only if parent(Q) locked by Ti
- (3) Items may be unlocked at any time
- (4) After Ti unlocks Q, it cannot relock Q

- Assume
  - Use an exclusive lock: $Lock_i(X)$ or $I_i(X)$
  - Consistent transactions
  - Legal schedule

Tree protocol

- Is used typically for B-tree concurrency control



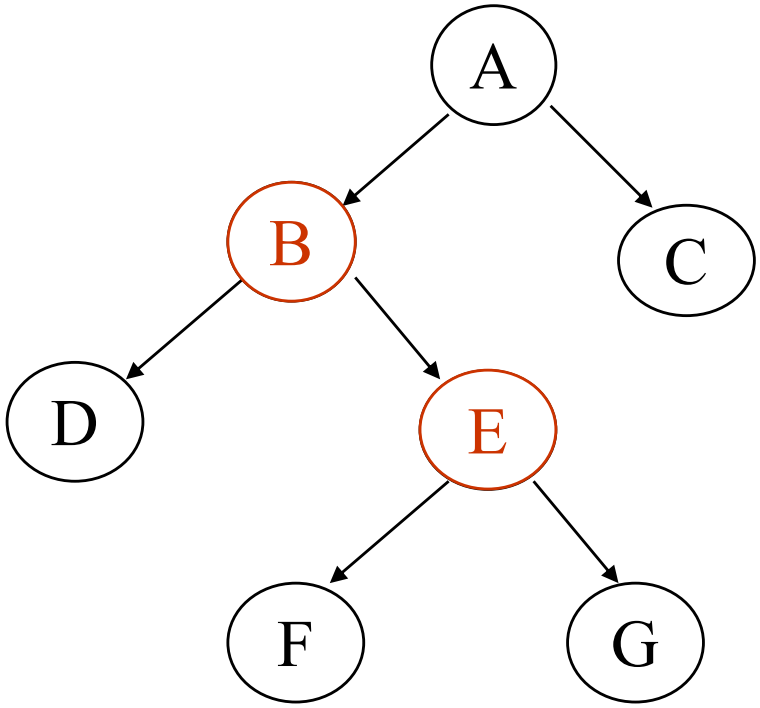
- During insert, do not release parent lock, until a certain child does not have to split

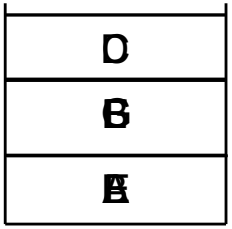
Example

T_1: l(A); r(A); l(B); r(B); l(C); r(C); w(A); u(A); l(D); r(D); w(B); u(B); w(D); u(D); w(C); u(C)

T_2: l(B); r(B); l(E); r(E); w(B); u(B); w(E); u(E)

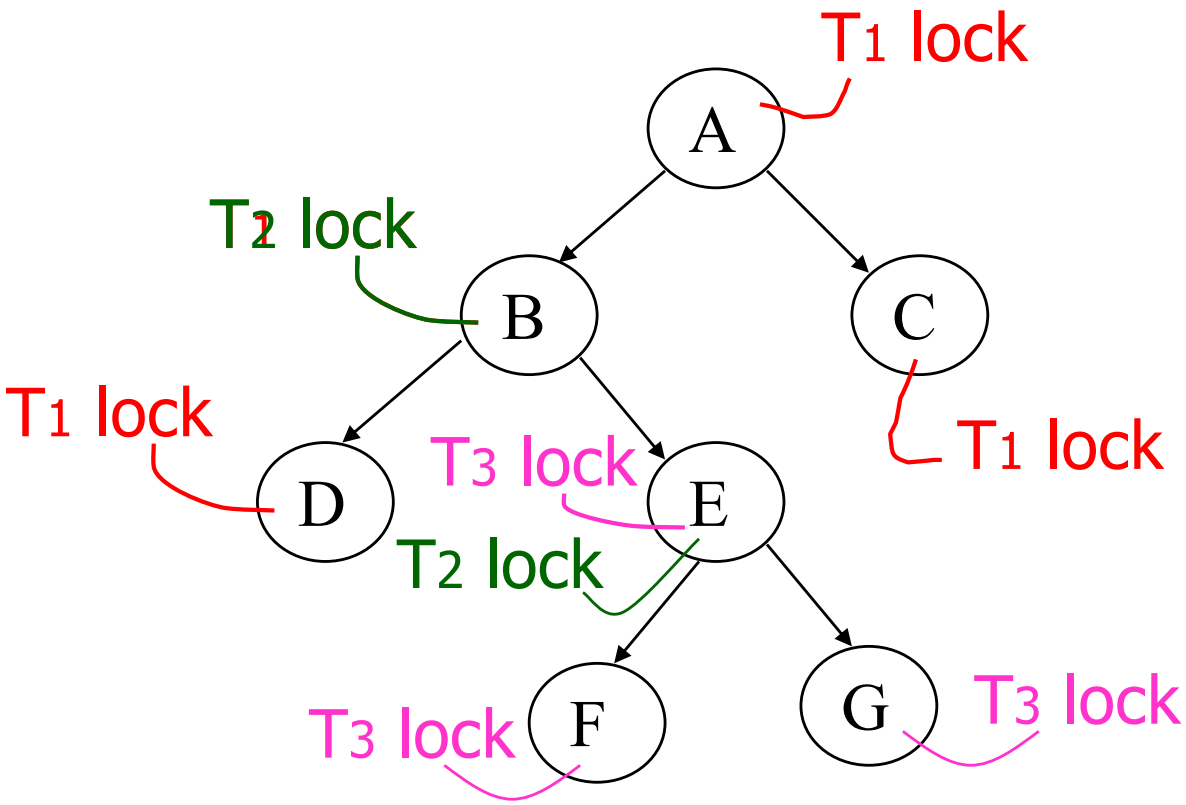
T_3: l(E); r(E); l(F); r(F); w(F); u(F); l(G); r(G); w(E); u(E); w(G); u(G)





Example

T_1	T_2	T_3
L(A); R(A)		
L(B); R(B)		
L(C); R(C)		
W(A); U(A)		
L(D); R(D)		
W(B); U(B)		
	L(B); R(B)	
		L(E); R(E)
W(D); U(D)		
W(C); U(C)		
	L(E)	
	↓	L(F); R(F)
	Wait	W(F); U(F)
		L(G); R(G)
		W(E); U(E)
	L(E); R(E)	
		W(G); U(G)
	W(B); U(B)	
	W(E); U(E)	



Schedule is serializable following a tree protocol

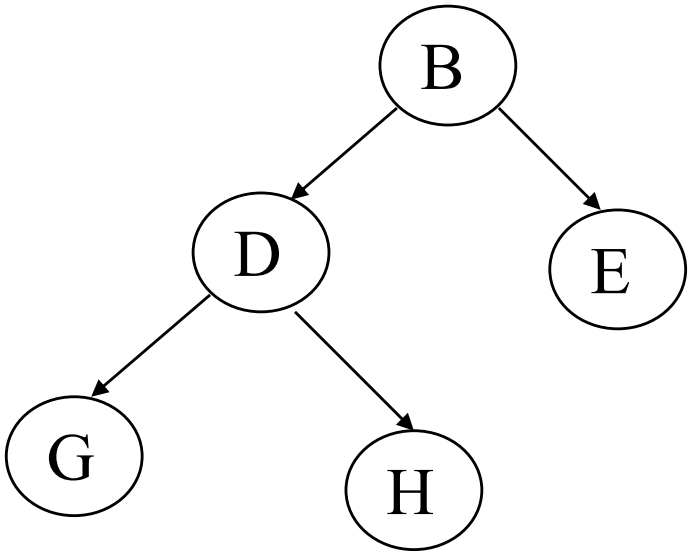
Example

T_1: l(B); l(E); l(D); u(B); u(E); l(G); u(D); u(G)

T_2: l(D); l(H); u(D); u(H)

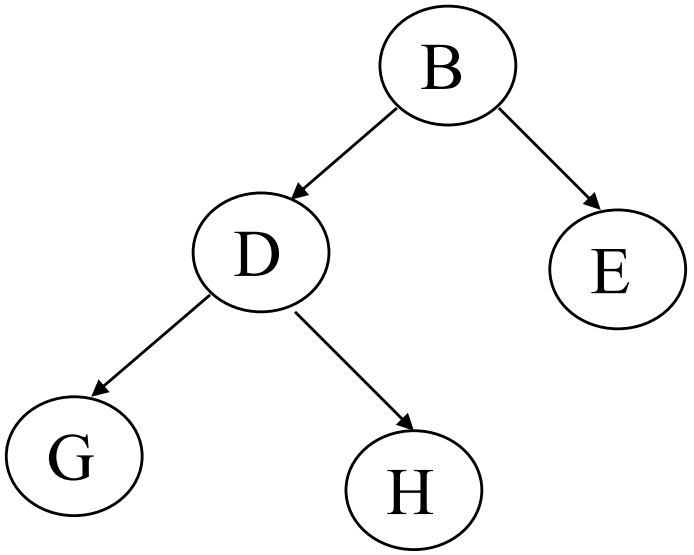
T_3: l(B); l(E); u(E); u(B)

T_4: l(D); l(H); u(D); u(H)



Example

T_1	T_2	T_3	T_4
Lock(B)			
	Lock(D)		
	Lock(H)		
	Unlock(D)		
Lock(E)			
Lock(D)			
Unlock(B)			
Unlock(E)			
		Lock(B)	
		Lock(E)	
	Unlock(H)		
Lock(G)			
Unlock(D)			
			Lock(D)
			Lock(H)
			Unlock(D)
			Unlock(H)
		Unlock(E)	
		Unlock(B)	
Unlock(G)			



Example

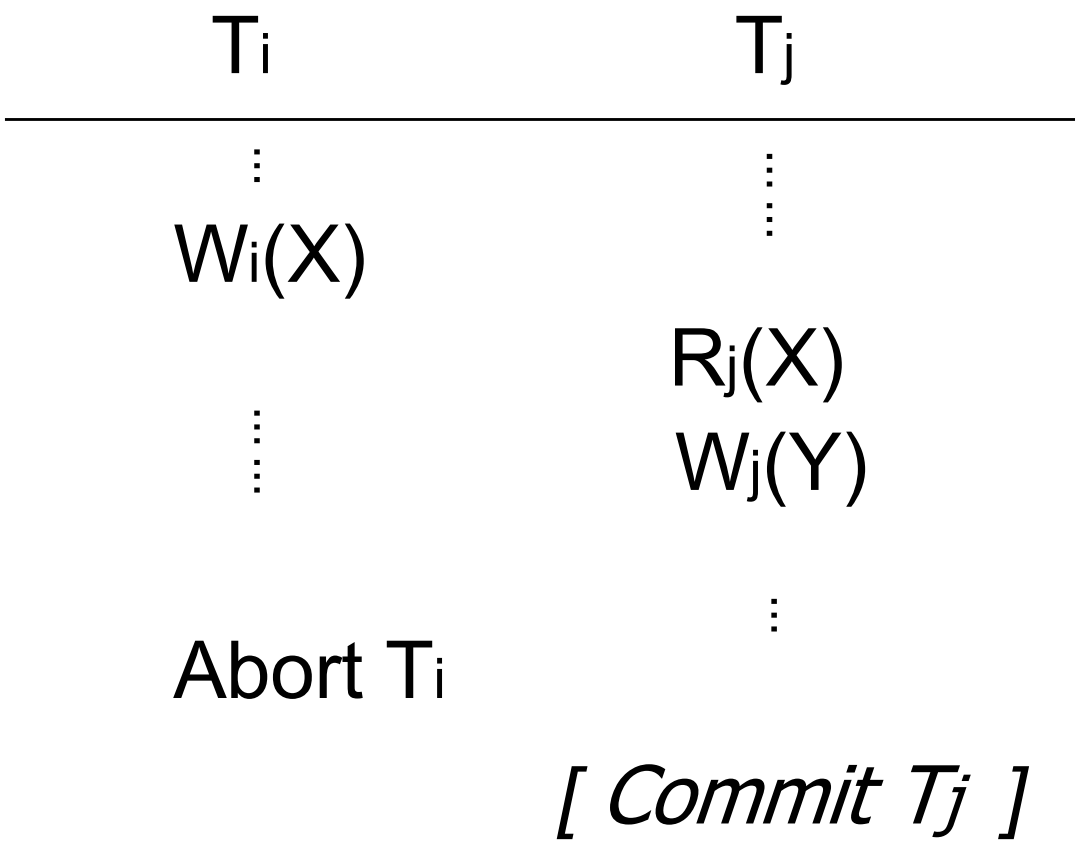
S	T_1	T_2	A	B
			25	25
	Lock(A); Read(A,t)			
	t:=t+100; Write(A,t)		125	
	Lock(B); Unlock(A)			
		Lock(A); Read(A,s)		
		s:=s*2; Write(A,s)	250	
		Lock(B)		
	Read(B,t); t:=t+100	*wait*		
	Write(B,t); **Unlock(B)**			
		Lock(B); Ulock(A)		
		Read(B,t); t:=t*2		
		Write(B,t); **Unlock(B)**		250

Example

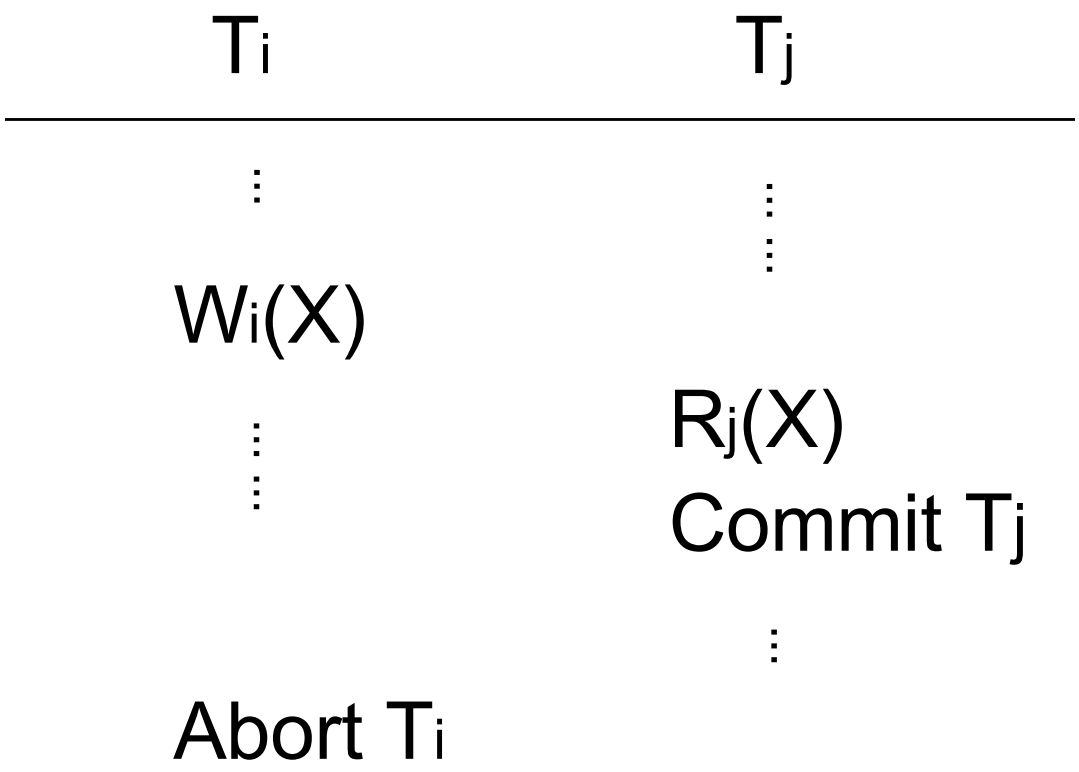
S	T_1	T_2	A	B
			25	25
	Lock(A); Read(A,t)			
	t:=t+100; Write(A,t)		125	
	Lock(B); Unlock(A)			
		Lock(A); Read(A,s)		
		s:=s*2; Write(A,s)	250	
		Lock(B) → *wait*		
	Read(B,t); **Abort;**			
	t:=t+100; Write(B,t);			
	Unlock(B);			
		Lock(B); Ulock(A)		
		Read(B,t); t:=t*2		
		Write(B,t); **Unlock(B)**		50

Consistency is violated, T2 must rollback

Cascading rollback



Not recoverable



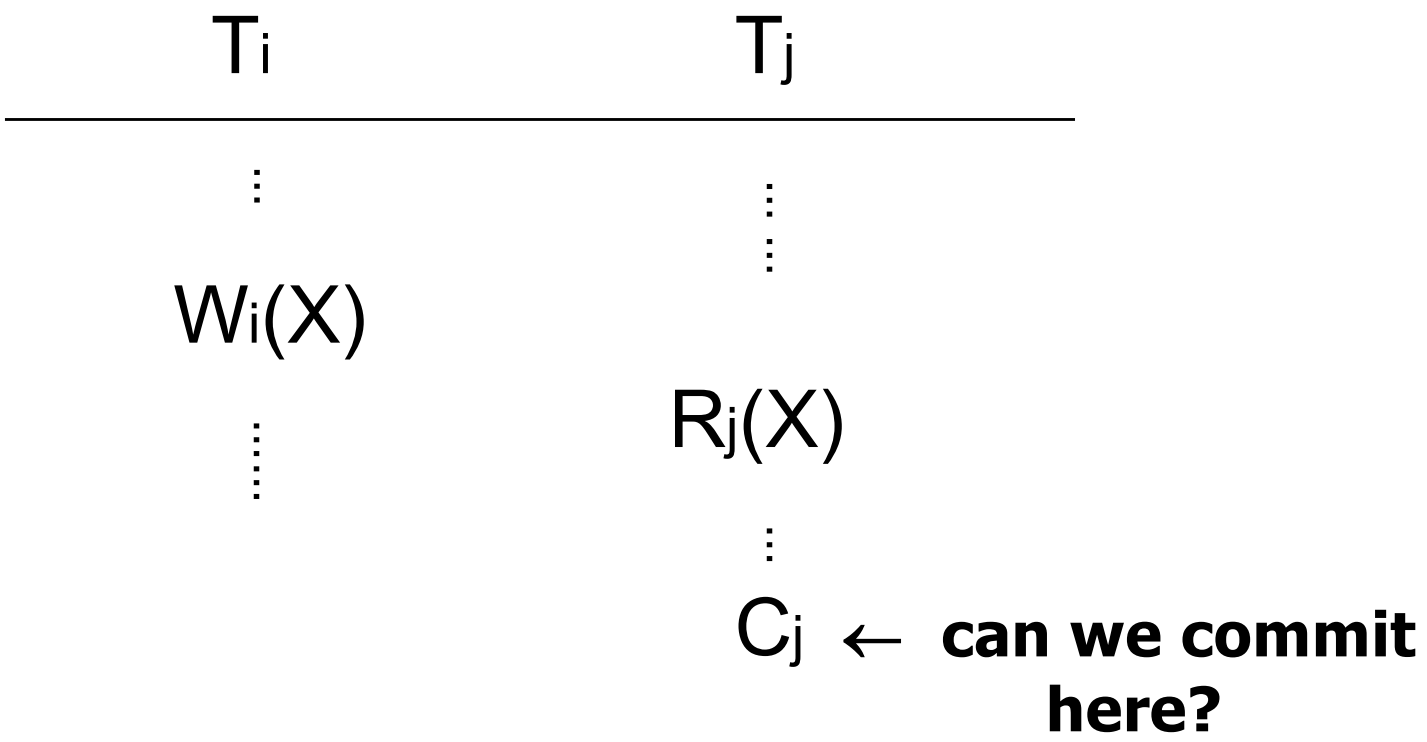
Discussion

- Problem
  - Schedule is conflict serializable
  - Ti ⟶ Tj
  - But not recoverable

- Need to make “final” decision for each transaction
  - **Commit decision:** system guarantees transaction will or has completed, no matter what
  - **Abort decision:** system guarantees transaction will or has been rolled back (has no effect)

- To model this, two new actions
  - C_i : transaction T_i commits
  - A_i : transaction T_i aborts

Discussion



Recoverable schedule

- T_j reads from T_i in S ($T_i \Rightarrow_S T_j$) if
  - (1) $W_i(X) <_S R_j(X)$
  - (2) $A_i \not<_S R_j(X)$
  - (3) If $W_i(X) <_S W_k(X) <_S R_j(X)$ then $A_k <_S R_j(X)$

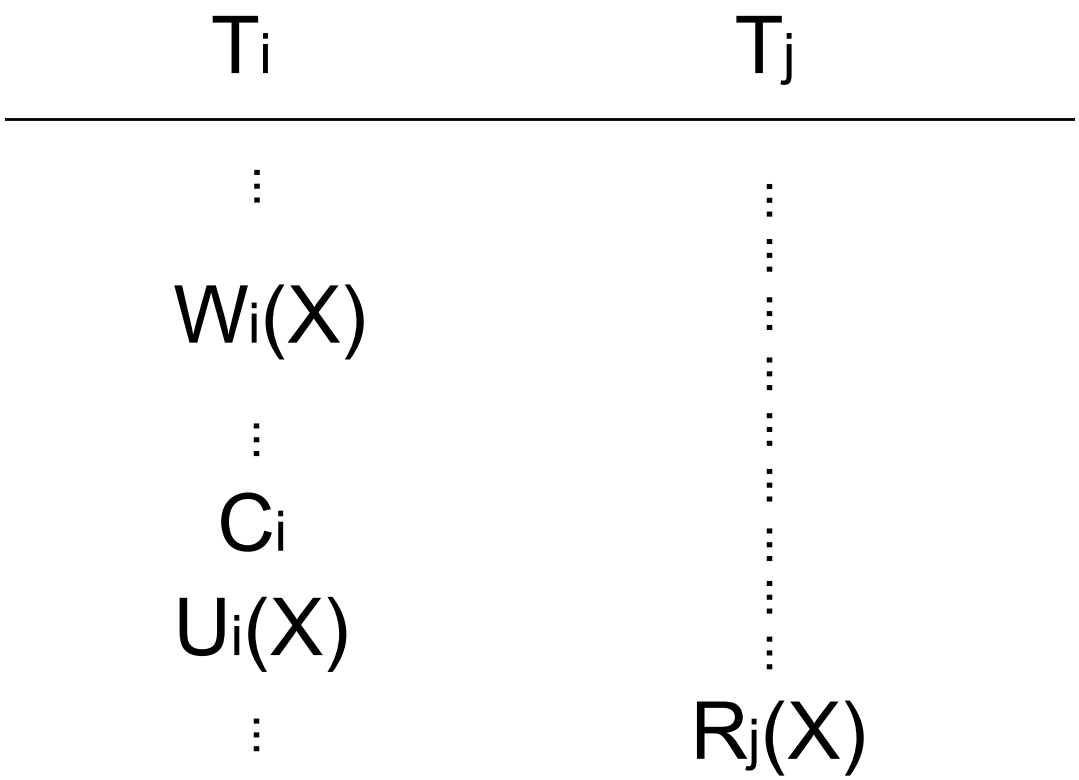
- Schedule S is recoverable if
  - Whenever $T_i \Rightarrow_S T_j$ and $i \neq j$ and $C_j \in S$
  - Then $C_i <_S C_j$

Note

- In all transactions, reads and writes must precede commits or aborts
  - If $C_i \in T_i$, then $R_i(X) < C_i$, $W_i(X) < C_i$
  - If $A_i \in T_i$, then $R_i(X) < A_i$, $W_i(X) < A_i$

- Just one of C_i, A_i per transaction

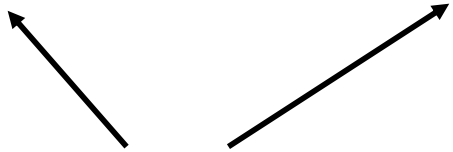
With 2PL



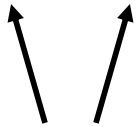
Hold write locks until commit ("Strict 2PL")

Example

S_1 : $w_1(A); w_1(B); w_2(A); r_2(B); c_1; c_2;$



T_2 reads after T_1 writes

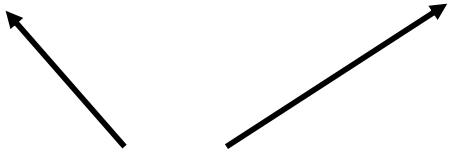


T_1 completes before T_2 completes

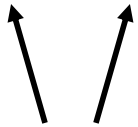
S_1 is serializable and recoverable

Example

S_2 : $w_2(A); w_1(B); w_1(A); r_2(B); c_1; c_2;$



T_1 writes before T_2 read

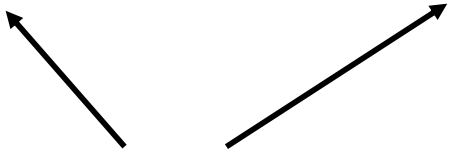


T_1 completes before T_2 completes

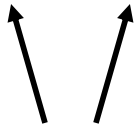
S_2 is not serializable but recoverable

Example

S_3 : $w_1(A); w_1(B); w_2(A); r_2(B); c_2; c_1;$



T_1 writes before T_2 reads



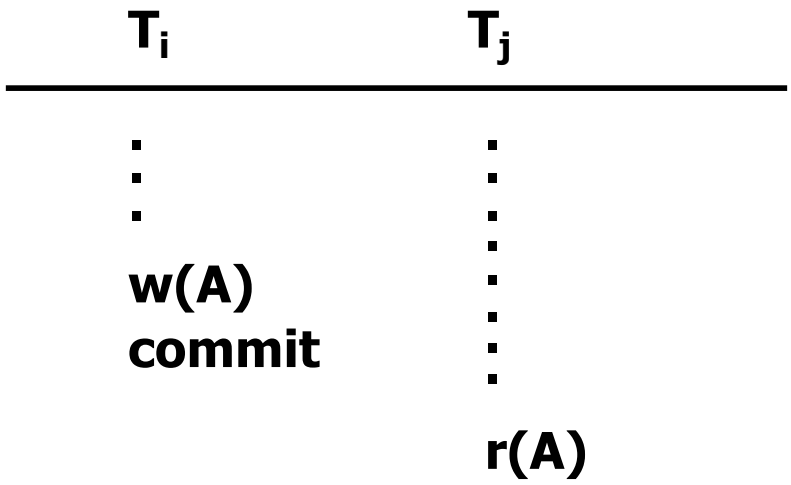
T_2 completes before T_1 completes

S_3 is serializable but not recoverable

Discussion

- We want to recover, sometimes we need to roll back
- But cascading rollback cannot happen

→ Cascadeless schedule



Transactions only read values that have been completed

Discussion

- Recoverable

  S : $w_1(A); w_1(B); w_2(A); r_2(B); c_1; c_2;$

- Avoids cascading rollback (ACR)

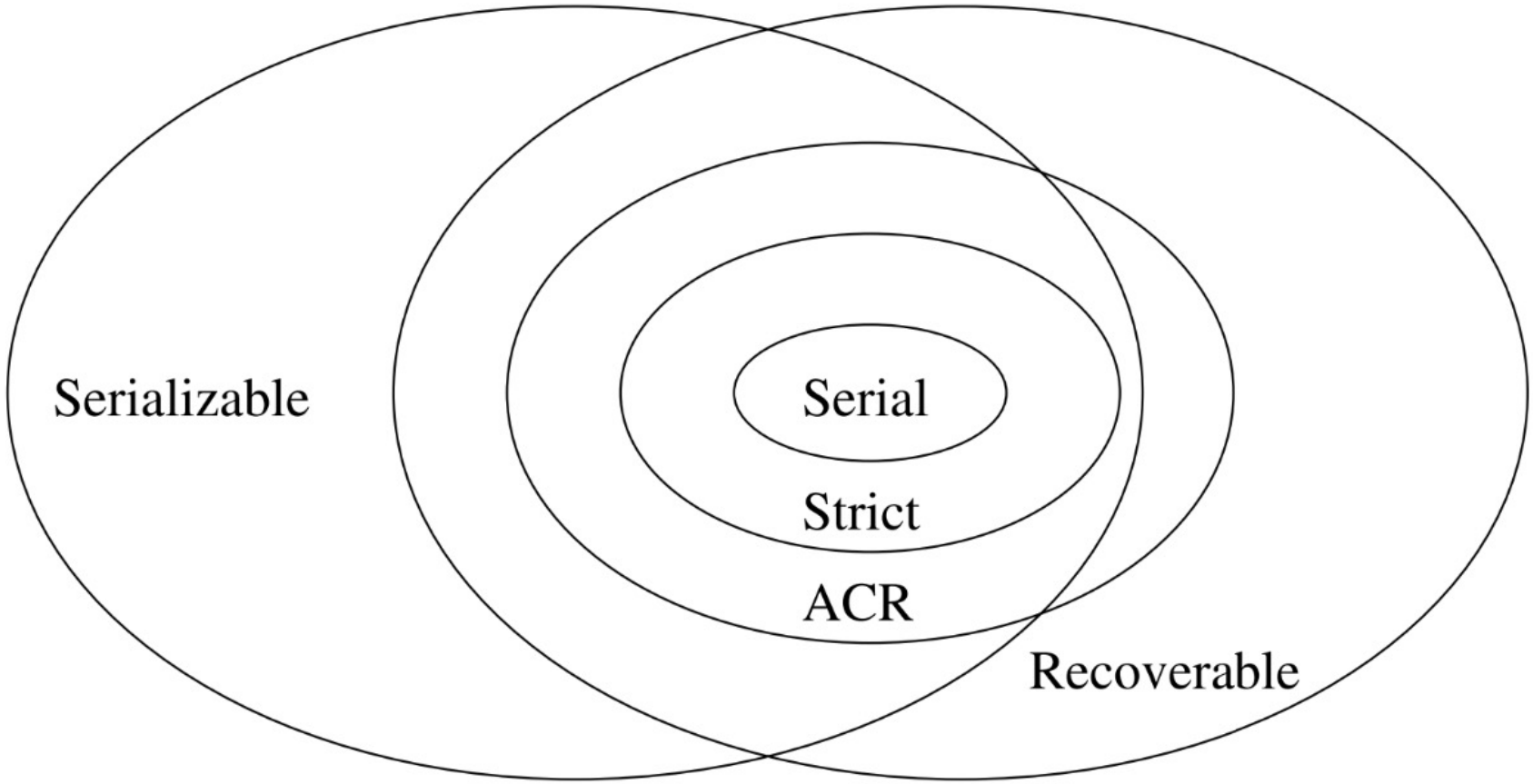
  S : $w_1(A); w_1(B); w_2(A); c_1; r_2(B); c_2;$

- Strict (≡ Strict 2PL)

  S : $w_1(A); w_1(B); c_1; w_2(A); r_2(B); c_2;$

→ Cascadeless schedules are recoverable

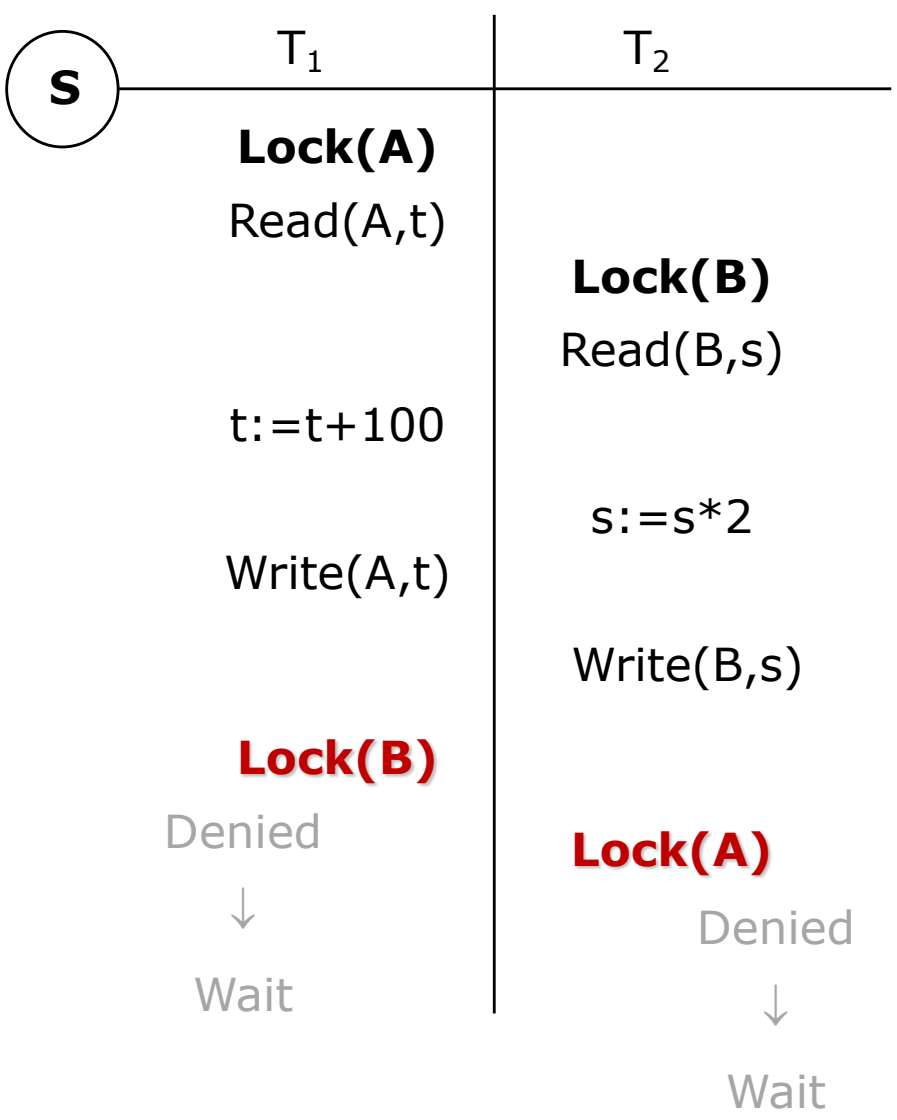
Recoverability & Serializability

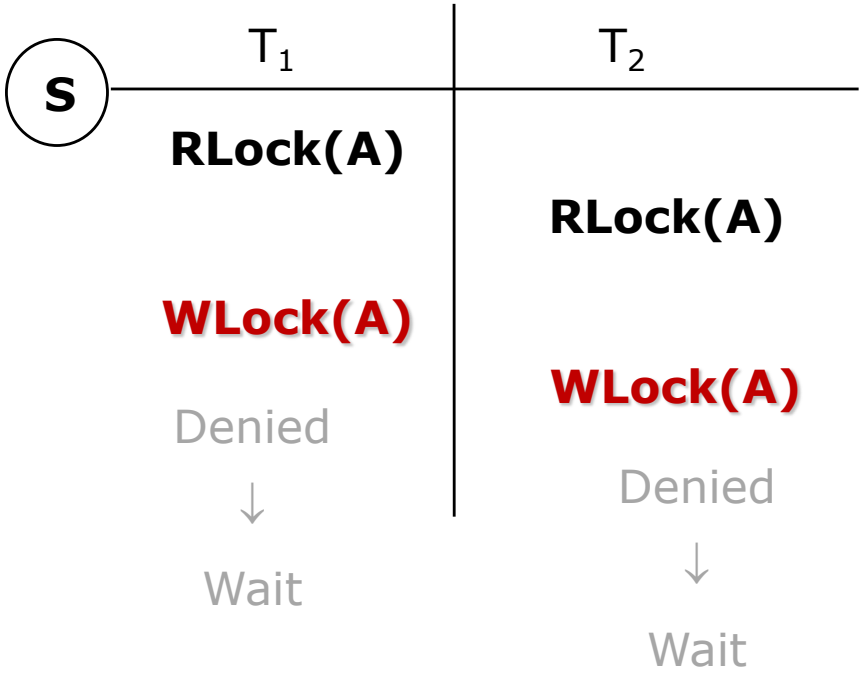


Content

- **More on transaction processing**
  - Cascading rollback
  - Recoverable schedule
  - Deadlocks : prevention & detection

Example


S
T1
T2
Lock(A)
Read(A,t)
Lock(B)
Read(B,s)
t:=t+100
s:=s*2
Write(A,t)
Write(B,s)
Lock(B)
Lock(A)
Denied
Denied
Wait
Wait


S
T1
T2
RLock(A)
RLock(A)
WLock(A)
WLock(A)
Denied
Denied
Wait
Wait

Resolving deadlocks

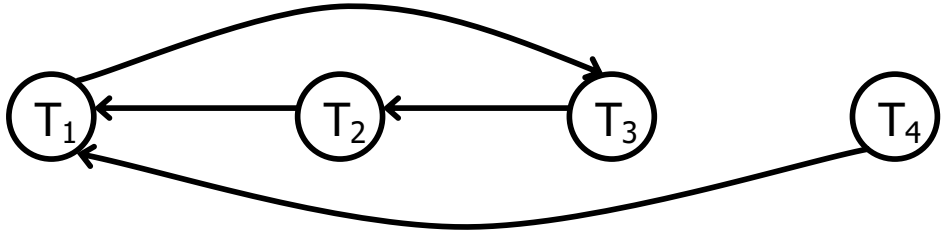
- Waif for graph
- Resource ordering
- Timeout
- Wait-die
- Wound-wait

Wait for graph

- Node
  - Transations that currently hold a lock or waiting for a lock
- Arc from node T_i to node T_j
  - T_j hold a lock on element A
  - T_i is waiting for a lock on A
  - T_i cannot get a lock on A unless T_j releases its lock on A
- If no cycles in the graph
  - Each transaction can complete
- Else
  - There is a deadlock

Example

	T_1	T_2	T_3	T_4
1	L(A); R(A)			
2		L(C); R(C)		
3			L(B); R(B)	
4				L(D); R(D)
5		L(A)		
6		↓	L(C)	
7		Wait	↓	L(A)
8	L(B)		Wait	↓
	↓			Wait
	Wait			



Resource ordering

- Order all elements $A1, A2, \ldots, An$
- A transaction T can lock Ai after Aj only if $i > j$

- If transactions request locks on elements in order
- Then there is no deadlock

- Note
  - Ordered lock requests not realistic in most cases

Example

T_1 : l(A); r(A); l(B); w(B); u(A); u(B);

T_2 : l(C); r(C); l(A); w(A); u(C); u(A);

T_3 : l(B); r(B); l(C); w(C); u(B); u(C);

T_4 : l(D); r(D); l(A); w(A); u(D); u(A);

Example

- Elements in alphabetical order

T_1 : l(A); r(A); l(B); w(B); u(A); u(B);

T_2 : l(A); l(C); r(C); w(A); u(C); u(A);

T_3 : l(B); r(B); l(C); w(C); u(B); u(C);

T_4 : l(A); l(D); r(D); w(A); u(D); u(A);

Example

	T_1	T_2	T_3	T_4
1	L(A); R(A)			
2		L(A)		
3		↓	L(B); R(B)	
4		Wait		L(A)
5			L(C); W(C)	↓
6			U(B); U(C)	Wait
7	L(B); R(B)			
8	U(A); U(B)			
9		L(A); L(C)		
10		R(A); W(C)		
11		U(C); U(A)		
12				L(A); L(D)
13				R(D); W(A)
14				U(D); U(A)

Timeout

- If a transaction waits more than L seconds
- Then roll it back

- Simple scheme
- But, hard to select L

Wait-die

- Transactions given a timestamp when they arrive, $ts(T_i)$
- T_i and T_j
  - T_i wants to lock on element A
  - T_j hold a lock on A
- T_i ***waits*** for T_j if $ts(T_i) < ts(T_j)$

- Else ***dies***

Example

	T_1	T_2	T_3	T_4
1	L(A); R(A)			
2		L(A)		
3		↓	L(B); R(B)	
4		**Dies**		L(A)
5			L(C); W(C)	↓
6			U(B); U(C)	**Dies**
7	L(B); R(B)			
8	U(A); U(B)			

Example

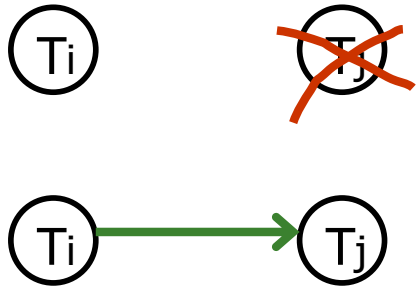
	T_1	T_2	T_3	T_4
1	L(A); R(A)			
2		L(A)		
3		↓	L(B); R(B)	
4		**Dies**		L(A)
5			L(C); W(C)	↓
6			U(B); U(C)	**Dies**
7	L(B); R(B)			
8	U(A); U(B)			
9		L(A); L(C)		
10				L(A)
11		R(A); W(C)		↓
12		U(C); U(A)		**Dies**
13				L(A); L(D)
14				R(D); W(A)
15				U(D); U(A)

Example

	T_1	T_2	T_3	T_4
1	L(A); R(A)			
2		L(A)		
3		↓	L(B); R(B)	
4		**Dies**		L(A)
5			L(C); W(C)	↓
6			U(B); U(C)	**Dies**
7	L(B); R(B)			
8	U(A); U(B)			
9				L(A); L(D)
10		L(A)		
11		↓		R(D); W(A)
12		**Waits**		U(D); U(A)
13		L(A); L(C)		
14		R(A); W(C)		
15		U(C); U(A)		

Wound-wait

- Transactions given a timestamp when they arrive, $ts(T_i)$
- T_i and T_j
  - T_i wants to lock on element A
  - T_j hold a lock on A
- T_i ***wounds*** for T_j if $ts(T_i) < ts(T_j)$
  - Tj rolls back and gives lock to Ti
- Else ***waits***



Example

	T_1	T_2	T_3	T_4
1	L(A); R(A)			
2		L(A)		
3		↓	L(B); R(B)	
4		**Waits**		L(A)
5	L(B); R(B)		↓	↓
6	U(A); U(B)		**Wounds**	**Waits**
7		L(A); L(C)		
8		R(A); W(C)		
9		U(C); U(A)		
10				L(A); L(D)
11				R(D); W(A)
12				U(D); U(A)
13			L(B); R(B)	
14			L(C); W(C)	
15			U(B); U(C)	

Discussion

- Wait-die & wound-wait
  - No starvation
    - Transactions repeat a process of (start, deadlock, rollback)
  - Wound-wait has less rollback than wait-die
  - Easier to implement than wait for graph
  - Sometimes rollback transactions not causing deadlock

- Wait for graph
  - Constructing a graph is complicated when database is distributed
  - Minimizes the number of times of rollback

Next lecture

- Recovery
- Read chapter 17 [1]